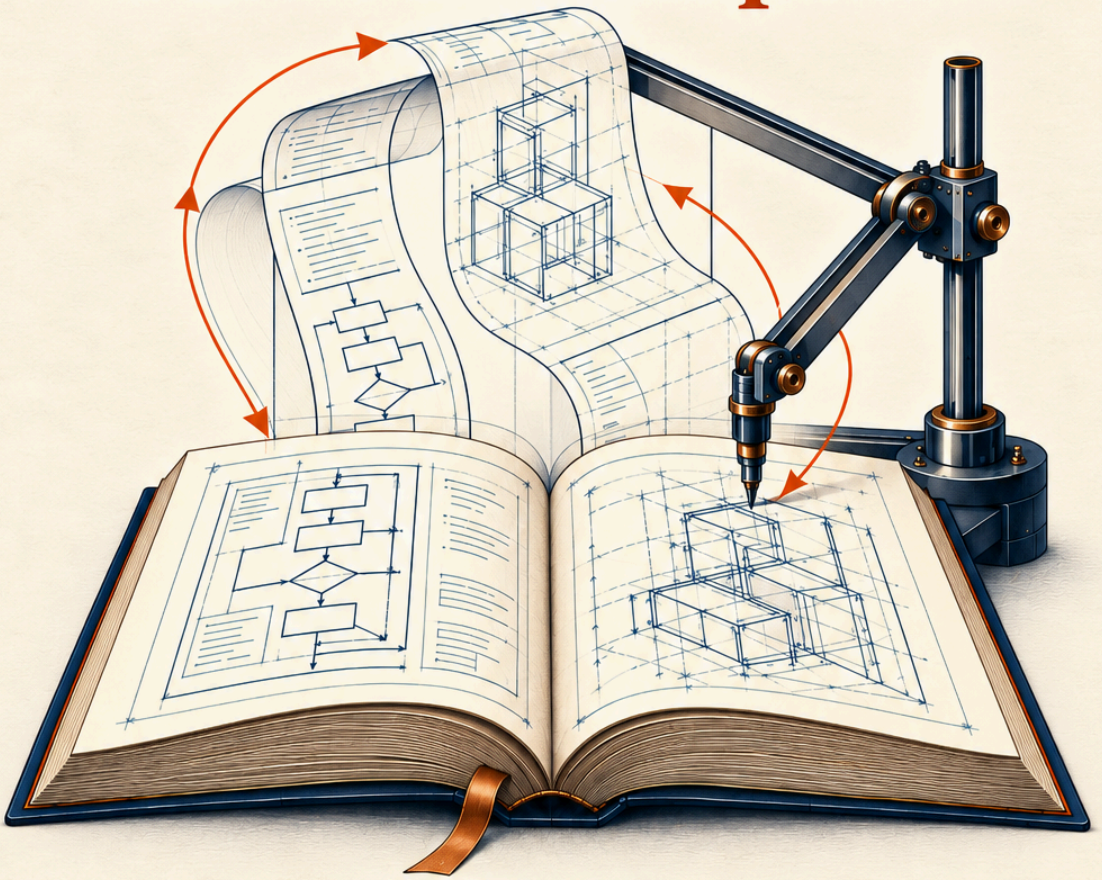
The Art of Building AI Coding Agents

A Human Companion



Bill Cox

Series: The Singularity as it Happened, Book 6

The Art of Building AI Coding Agents

A Human Companion (C# Edition)

The Singularity as it Happened — Book 6

Bill Cox

Copyright © 2026 Bill Cox. All rights reserved.

The Art of Building AI Coding Agents: A Human Companion (C# Edition)

The Singularity as it Happened — Book 6

To Caitlin Cox,
the sweetest, the cutest, and the smartest daughter.

Contents

Preface 1
Chapter 0: The Perpetual Machine 5
Chapter 1: One Loop, Sixty Billion Dollars 10
Chapter 2: One Log, Three Vendors 16
Chapter 3: Six Tools, Ninety-Two Percent of an AI Coding Agent 29
Chapter 4: Jobs, or Why a Tool Call Is a Process You Supervise 35
Chapter 5: Two Seams and a Loop 41
Chapter 6: Streaming, or the Same Answer in Pieces 48
Chapter 7: Everything Is an Artifact 53
Chapter 8: Build Your Dream GUI 59
Chapter 9 — Skills 64
Chapter 10 — Persistence and Replay 70
Chapter 11 — MCP Tools and a Browser That Can Answer 76
Chapter 12: The Agent Sees Its Working Interface 83
Chapter 13 — A listener who can only hear 90
Chapter 14 — The world's best context engineering, before breakfast 95
Chapter 15: The Invisible Invoice 101
Chapter 16: Leaving Anthropic for OpenAI 107
Chapter 17: Giving the Agent the Web, and a Status Line 113
Chapter 18: The Sandbox 119
Chapter 19: The Ensemble 126
Chapter 20: Managing the Swarm 133
Chapter 21: Forgetting on Purpose 140
Chapter 22: Auto-Recall 147
Chapter 23: While the Cache Is Broken — Measure Before Taking the Detour 154

Chapter 24: Gateway 159
Chapter 25 — Goals 166
Epilogue: What we can rebuild 171

Preface

A coding agent makes a surprising number of decisions about how you are allowed to work. It decides what you can see while it is thinking, when you can interrupt, what it remembers, and which actions require your approval. You may like its code and dislike the working relationship. Changing the prompt can help. Sometimes the thing that needs changing is the program around the model.

Bill's reason for building his own agent begins there. He wanted an agent suited to the way he thinks and works, with decisions he could understand and change. The ambition behind this book is to make that knowledge available to someone else. A coding agent is software, not a privilege reserved for whoever can finance a model. You can understand its construction, own the machinery you build, and decide which parts deserve to survive.

The prize is not an agent declared best for everyone. Bill's preferred agent would be wrong for another engineer in ways neither could predict in advance. The useful inheritance is the ability to tell which choices protect a necessary property and which express a preference. Otherwise, building your own agent just gives you a more intimate relationship with someone else's assumptions.

Independence has a boundary

Owning that machinery gives you real control. You can decide how tools are exposed, where authority is enforced, what history is retained, and how a human steers work already in progress. When the behavior is wrong, there is somewhere to look besides the prompt. When a design no longer fits, you can change it.

It does not make the model yours. A hosted model still brings a provider, an API, operating limits, and policies you do not set. Code sent to that service has left your machine, even if every line of the agent itself was written locally. A local model changes those dependencies; it does not remove the need to understand the model's limitations or secure the surrounding system.

Nor does an adapter make providers interchangeable. An adapter can translate message formats. It cannot make two models exercise the same judgment. Switching models means evaluating behavior as well as getting requests accepted.

That is a more useful meaning of independence than having no dependencies: knowing where they are, choosing them deliberately, and retaining enough control to respond when they change. The agent becomes a system you can reason about rather than a service whose habits you can only accommodate.

Keeping the expensive part

Bill reports spending a couple thousand hours building coding tools with language models. This companion is meant to transfer the experience behind that work to a human who can lead it forward. It assumes you already know how to get substantial work done with AI. The missing ingredient is not a tutorial on

prompting. It is the history of this particular machine: why its boundaries exist, what tempting alternatives cost, and which conclusions remain open to correction.

An implementation preserves the choices that won. It is much less reliable at preserving why they won. Once an awkward interaction has been replaced by a clean design, the reason for the design can disappear with the awkwardness. The next capable engineer can then remove an apparently unnecessary distinction and rediscover its purpose at full price.

The instructional book tackles that problem by making the knowledge precise enough for another agent to build from. This companion tackles the human side: understanding enough to judge the result, alter the design, and carry responsibility for what follows. Neither is intended to preserve Bill as the person every consequential decision must pass through.

His opinions matter because they come from sustained work on the problem. They are not exemptions from explanation. A successor needs to be able to disagree usefully, including recognizing when a lesson was right for the circumstances that produced it and wrong for the circumstances now at hand.

A book that can be corrected

The original ambition goes beyond documenting one implementation. If the knowledge can survive a rebuild, a book can become part of the working machinery of a project. Experience with the agent can expose a mistake in the teaching; correcting the teaching can help the next implementation avoid it. Eventually the agent may become the principal tool used to improve both itself and the book.

That is an ambition, not a completed perpetual-motion machine. A newer model does not guarantee a better implementation. A passing check does not make the requirement wise. Writing a decision into a chapter does not prevent it from becoming technical debt. The useful possibility is that explanations and checks make the decision easier to examine, correct, and carry into subsequent work.

This is why the book needs an independent reader, not a disciple. Understanding the design includes being able to spot where the explanation outruns the evidence. Chapter 0 begins with what went wrong when Bill tried to transfer the knowledge, and what those failures revealed about the work of preserving it.

The knowledge and direction behind this companion come from Bill. AI assistants draft and review its prose under his direction; that does not imply that he has personally reviewed every passage. The aim is to make his experience available for examination, including correction, rather than put a polished surface over it.

The agent should become yours in the consequential sense: you understand what it does, can explain why it does it, and can decide when it ought to do something else.

Chapter 0: The Perpetual Machine

An experienced engineer can recognize the parts of an AI coding assistant: an HTTP client, a conversation, some tools, a terminal, perhaps a browser. None looks especially mysterious. Then a tool hangs, a second agent starts working on the same problem, and the conversation grows too large to send. Suddenly the hard questions sound familiar. Who owns this operation? Which result is authoritative? What can be reconstructed after failure? Who is allowed to stop what?

This book is about those questions. Ensemble is the running example: a coding agent built as a library, with front ends and external services around it. The model is important, but it is not the application. Much of the interesting engineering happens before the model receives a request and after it returns an answer.

There is a second experiment underneath the first. The system is described in an executable codebook: explanations, architectural constraints, a construction sequence, and checks that another AI can use to build it. This companion carries the engineering knowledge without the assignments, exact schemas, and grading machinery. A reader can understand the design here and consult the corresponding codebook chapter when an exact contract matters.

The distinction is useful because an instruction precise enough to execute is not always a pleasure to read. A schema must enumerate fields. An explanation needs to tell you why the schema has that boundary at all.

The second program

Writing a small assistant becomes interesting when it starts helping to write its successor. The current application supplies tools and working context while the next one is built. Defects discovered during construction can become changes to the implementation, but also corrections to the book that taught it.

Bill calls this the codebook's larger ambition: preserve enough engineering knowledge that the source can be rebuilt, rather than preserving only the latest source. The analogy is a compiler, with an important qualification. An AI executing prose is not deterministic. It can misunderstand, drift, or produce a plausible implementation with the wrong ownership model. Checks and expert judgment remain part of construction.

A source repository answers what the current implementation does. A good codebook also explains what must remain true when the implementation changes, and why. That distinction becomes valuable when an apparently helpful simplification removes the thing that made the system work.

The human companion has a different job again. It should make the architecture available to someone who does not intend to reproduce every exercise. An engineer reading on a train should leave with useful ideas about event histories, long-running work, ownership, cache accounting, context management, and delegated computation. No grader needs to accompany the train ride.

The architect is still present

Bill has used Go successfully in CodeRhapsody while supervising the assistant throughout the work. He listens to its reasoning and corrects architectural drift as it happens. Autonomous execution

of a codebook changes that arrangement. The instructions and implementation language must support the intended design without the same continuous rescue.

The failed Go execution made the distinction expensive. Bill discarded twenty-one chapters after Astra spent thirty hours unable to complete Chapter 21. He also found a problem in the instruction setup: the architectural rules had been presented as file content rather than installed as governing instructions. It would be convenient to turn that history into a controlled language experiment. It was not one. It exposed a failure of implementation and instruction delivery, and made the cost of fighting a language's familiar idioms impossible to ignore.

The C# edition therefore makes the model unusually explicit. Common declares the shared vocabulary. Functional modules implement it. Objects retain their real parent relationships. Relationship operations update both ends. Logical destruction is complete and intentional. These are Bill's longstanding engineering rules, not conventions invented to appease a language model.

C# is useful here because it accommodates those rules directly. Partial types and members separate declarations from implementations at compile time. Ordinary references can represent a graph without a second graph of handles or interface wrappers. Garbage collection reclaims unreachable storage, including cycles, while explicit destruction repairs the application's relationships. Familiar HTTP, JSON, and concurrency libraries keep the infrastructure ordinary.

The language does not enforce every architectural rule. All the partial pieces compile into one assembly, so directory-level dependency discipline still requires review. The gain is that following the design does not require runtime machinery whose sole purpose is connecting a declaration to its body.

Follow the ownership

The book begins with one conversation because a small program makes authority visible. A request contains the conversation so far. A response contributes text and measured usage. Failed operations must not leave a half-published turn behind.

Typed history then separates what happened from the next provider request. Tools introduce effects. Jobs give slow effects a lifetime that does not depend on a caller's patience. The actor gives conversation state one owner while requests, tools, and users remain active around it.

Once those boundaries are sound, interfaces become consumers rather than rival owners. Streaming improves observation without becoming a second answer. A GUI can stop without destroying the conversation. Skills can change instructions and available operations. Persistence can reconstruct committed facts instead of restoring a heap of incidental state.

The later chapters apply the same reasoning at larger scales. Memory replaces a working context without erasing history. Recall brings selected earlier material back without repeatedly injecting the same snippets. Children have explicit parents and completion contracts. Gateway introduces a network boundary without changing who owns local work. Goals preserve an objective and create an opportunity to resume, while leaving authority and judgment where they belong.

This sequence is deliberately cumulative. An agent that cannot distinguish a wait from a cancellation should not yet supervise a fleet. A memory system that cannot distinguish an archive from a working view should not decide which parts of a conversation may disappear.

Receipts, not legends

The project includes historical incidents, local tests, browser exercises, and bounded live-provider runs. They answer different questions. A fake can demonstrate that a cancellation reaches a transport. A live request can show that a service accepted particular credentials and bytes. Neither alone establishes that a complicated assistant is generally reliable.

The companion uses those experiences where they explain a decision. It does not reproduce the score tables or ask a single successful run to support a larger claim. The detailed contracts and execution record remain available in the codebook.

The current implementation sequence is complete through Goals. The crossover into the new running system is a separate, ongoing piece of work, and its account is reserved for the final chapter. There is no need to manufacture an ending in advance.

For now, start with the least impressive-looking part of the system: a paid HTTP request, a list of messages, and a loop.

Implementation details: codebook Chapter 0.

Chapter 1: One Loop, Sixty Billion Dollars

A model provider has done something expensive and difficult. Calling its model should not require pretending that the application has done the same thing.

At the beginning, the assistant is an HTTP client. It sends a model name, instructions, a conversation, and an output budget. It receives typed content and usage. It appends a completed exchange and does it again. The request is small enough to inspect, which is an advantage worth preserving as long as possible.

There is no agent framework yet. There are no tools, background jobs, skills, or memory summaries. Adding them before understanding the first request would make it harder to discover which layer is responsible for a failure.

A conversation is resent

The provider does not retain this program's conversational state between ordinary requests. The application sends the relevant history again. A question about the second-highest mountain only makes sense if the earlier question and answer are included.

That yields a deceptively effective data structure: a retained list of role-and-content values. The library exposes a snapshot rather than allowing callers to mutate that list. The first chapter

deliberately uses a narrow alternating user/assistant conversation. It is a useful local contract, not a claim that the provider's complete protocol forbids every other shape.

Instructions travel separately from that list. So do the requested model and the output budget. Keeping them distinct prevents a tempting early mistake: treating the exact JSON used by one provider as the application's permanent model of a conversation. The next chapter will replace the small message representation, but there is no reason to anticipate its entire vocabulary in this one.

One wire detail deserves attention immediately. A request may accept plain text where a response returns an array of typed blocks. Reading the first block's text works until the provider returns two text blocks, or a block that is not text at all. Walking the array costs almost nothing and avoids making a successful shortcut into an architectural assumption.

The same caution applies to model names. A model identifier remembered from training data can look plausible and still be obsolete. Querying the provider's model catalogue is less glamorous than guessing the newest name, and considerably more dependable. Printed identifiers age; discovery is the durable lesson.

Put the facts where they belong

The application begins with a few concrete objects. Ensemble owns its Agents and the application logger. Each Agent owns its configuration, conversational state, and the Engine that communicates with the provider. The Engine reaches configuration through its Agent and logging through the Agent's Ensemble parent. Those paths are part of the shared vocabulary, rather than services passed separately into every constructor.

Common contains the type declarations, persistent fields, relationships, and callable signatures. Functional directories supply the matching partial implementations. C# 14 allows the constructor declaration to participate in that separation too. The compiler combines the parts into one type in one assembly; no wrapper object connects the declaration to its implementation.

This is source organization, not a dependency-injection trick. The same object has the same fields on both sides. Implementations use Common's vocabulary instead of reaching into another functional module's private helpers. Application composition is the place allowed to construct and connect the system.

The discipline matters even in a small program. If the Engine needs the Agent's configuration, a parent path expresses why it is entitled to reach it. A copied configuration reference may look harmless, but it creates another fact to preserve when ownership changes. If the path is awkward, the model deserves attention before another constructor parameter is added.

Relationship-bearing objects also know their parents. A constructor registers a new child through the parent's add operation. That operation updates the parent's collection and the child's back-reference together. Removal does the reverse. Validation happens before either side changes, so refusal leaves a consistent model.

A message value is different. It does not need to know every list that contains it. Immutable value-like data can be shared without inventing ownership relationships solely to reclaim memory. The distinction prevents a sound rule about model relationships from turning into back-pointer bureaucracy for every string and vector.

Memory reclamation is not destruction

C#'s garbage collector can reclaim a disconnected cycle. It cannot decide what removing an Agent means to this application.

Logical destruction closes resources, recursively destroys strongly owned children, removes weak child relationships without destroying their objects, and detaches from every parent through the same complete relationship operations used elsewhere. An object's finalizer is not the place to repair the model. The program needs those postconditions when destruction returns, not whenever memory happens to be collected.

Multiple ownership makes this especially important. A child may legitimately have more than one parent. The model should represent that relationship rather than force it into a tree for convenience. Cleanup must then avoid destroying the same child twice, while still removing every relationship that pointed to it.

This may sound heavy for a first HTTP request. The alternative is to build the system on a simpler-looking ownership fiction and rediscover the missing relationships when the first fleet is introduced.

Publish a completed exchange

A request attempt and a completed conversational turn are not the same event.

The user message is staged for the request. The reply is decoded and validated. Only a successful answer publishes the user/assistant pair into the retained conversation. A malformed reply must not leave the user half of a failed exchange behind and make the next call violate the program's own conversation contract.

Usage requires a separate decision. The provider may report measured consumption even when the answer is unusable. Those tokens were still consumed. Keeping conversation publication transactional does not justify forgetting their cost.

The implementation therefore distinguishes valid usage from valid answer content. Missing usage is not measured zero. Cumulative counters are calculated with checked arithmetic before either total is published, so an overflow cannot update one half of the accounting and leave the other half old. Error diagnostics should identify the failing operation without leaking credentials or reflecting a remote error body indiscriminately into logs.

These are small choices, but they establish an important habit: each fact has its own acceptance boundary. The conversation, the bill, and the diagnostic are related without being the same transaction.

Slowness is not permission to retry

The first loop does not hide a failed paid request behind automatic retries. A rate limit is reported. A malformed response is reported. The application does not silently turn one user action into several generations and hope one works.

The request's lifetime is likewise explicit. A default whole-request timeout can abort an otherwise valid long generation. The Engine removes that accidental deadline and propagates the caller's cancellation through both header and body waits. Connection establishment may have a separate bound; abandoning a working generation is another decision.

Retries and supervision arrive later with enough identity and state to explain what they mean. Implementing them early would obscure a very useful observation: exactly one request was

attempted, and this is how it ended.

The growing bill

The first cost lesson follows directly from resending history. A short question near the end of a conversation can be expensive because its request includes everything retained before it. The current prompt is not the unit of input cost. The assembled request is.

Measured input and output totals make that behavior visible from the first turn. Later chapters will add caching, normalization across providers, and context reduction. None can repair accounting that began by treating missing numbers as zero or by reporting only the latest user sentence.

The tiny client is useful because there is nowhere for these facts to hide. You can inspect the outgoing conversation, the returned blocks, the retained pair, and the measured usage. When the assistant becomes more capable, those four observations should remain understandable.

The next step is not a larger loop. It is a better representation of what the loop knows.

Implementation details: codebook Chapter 1.

Chapter 2: One Log, Three Vendors

The first interface in CodeRhapsody looked like preparation for a second vendor. Its methods took Claude messages.

In the history preserved by the source chapter, Bill's first client supplied both the implementation and the vocabulary of the interface extracted from it. When Gemini arrived, the new client had to translate its conversation into Claude's terms before doing anything useful. Copying the existing client was easier. OpenAI acquired another copy, and fixes acquired several places to be forgotten.

The eventual replacement separated a common conversation model from vendor renderers and parsers. It reduced duplication, but the big-bang migration left capabilities behind in the old clients; the source records broken audio attachments in the replacement. The better boundary did not make an incomplete replacement complete.

Chapter 1 established a simple conversation with one owner. The next step preserves that behavior while removing the first provider's authority to define what a conversation is. There is still one agent, not a different agent for each API.

The record, the working view, and the request

Three things commonly called “history” need different rules.

History records what happened. **Context** is the current conversational meaning derived from that history. A **request** presents that context in the form a particular provider can accept. History must preserve an original tool result; context may replace it with a redaction stub; a request may move that stub next to its call. None of those operations licenses rewriting the other two.

Agent owns History and Engine. History appends events and reduces them into context. Engine renders requests, handles the exchange, and parses responses into neutral events, which Agent records. A parser does not edit context behind the reducer's back, and Engine does not keep a competing transcript.

An ordinary turn starts with a recorded human message. Reduction adds its content to dialogue and makes input pending. Engine builds a request from that context and configuration. Recording the send marks the turn in flight; the already-built request goes out. A parsed response returns through the event log and the same reducer. A reply without calls finishes the turn. A reply containing tool calls leaves tools pending; their recorded results eventually make input ready for another exchange.

Asking for a tool and running it are separate facts. The model's reply preserves text and calls together, in order. A later dispatch event says execution began without adding another copy of the call to dialogue. The return supplies the outcome. Chapter 2 can represent and replay these facts without executing tools yet. That capability belongs to the next chapter, not to a hidden scheduler inside a vendor adapter.

The append-only log is readable JSON lines, ordered by sequence number rather than wall-clock time. Timestamps help investigation; they do not settle ordering when clocks tie or disagree. Redaction and failure add events instead of editing earlier ones. The send event identifies the destination without storing a second authoritative request body.

That last economy has a limit: replaying conversation state does not necessarily reproduce every byte of a historical HTTP request. Rendering also depends on configuration and implementation version. A current configuration cannot stand in silently for the one used at the time.

Replay must not discover new facts

Reduction depends only on the previous context and the next event. It does not consult the network, current time, Engine's private state, or a second copy of history. Replaying a log neither sends another request nor reruns a tool.

The state machine distinguishes idle, input pending, in flight, and tools pending. A known event that has no applicable transition leaves the state unchanged rather than crashing replay. An unknown event kind is different: the loader refuses it before reduction. Ignoring an unfamiliar fact would make a readable-looking context with unknown omissions.

Input remains recordable while work is in flight. Arrival is not permission to cancel, redirect, or restart that work. Those later decisions need explicit, replayable meaning rather than guesses from an error message or the presence of old tool results. Chapter 2 establishes the recording boundary; the interrupt protocol comes later.

Temporary instructions expose why ordering matters. An ephemeral System contribution goes into a pending slot, not permanent dialogue, and its arrival alone does not start a request. A newer contribution replaces it; an empty update clears it. All arrival events remain recorded. Neither operation changes the stable primary instructions.

Rendering cannot consume the slot: rendering the same snapshot twice should give the same request. The send event consumes it, even if the exchange subsequently fails. Therefore Engine builds from the snapshot that still contains the instruction, records the send, and uses that built request. Re-rendering after the send would lose the instruction before delivering it. Keeping it after a failed exchange would let stale advice leak into an unrelated turn. This is a one-request context policy, not a promise of exactly-once delivery across a network.

The source records another confusion between history and current state: a test classified a request as being in a tool loop whenever it found any tool results. Full dialogue was being resent. After the first tool use, the test could never recognize a fresh turn again. Current activity belongs to the turn state and the relevant end of the conversation, not a search for something that once happened.

Values without duplicate owners

The C# model makes these boundaries physical. Context entries contain ordered, immutable, value-like parts rather than mutable bags of loosely named fields. Text, tool calls, tool results, blob references, opaque replay material, and redaction stubs retain different meanings even when a provider eventually puts several of them in one JSON object.

Constructors copy mutable inputs. Part arrays are not exposed for editing; tool arguments remain privately owned JSON rather than caller-owned dictionaries or JSON trees. Accessors cannot hand out a route back into recorded state. A renderer is allowed to rearrange its output, not mutate the context it received. Sharing genuinely immutable values is harmless; sharing an array that someone still owns and can edit is not immutability.

Value-like does not mean that every part must be a C# value type. A class can have value-like semantics and need no back-pointer to every entry containing it. Garbage collection reclaims it when it is unreachable. History and Engine are different: they are relationship-bearing children of Agent, each retaining its parent reference. Parent add and remove operations establish and clear both ends of the relationship; destruction releases owned resources rather than leaving a half-detached child for some other method to repair.

Common declares the shared types, persistent state, relationships, and explicitly accessible partial operations. Matching C# 14 partial type sections put the bodies in their responsible modules, compiled into one assembly. There is no second object hiding behind an injected service field to connect a declaration to its implementation. Functional modules use the Common vocabulary, not one another's private helpers.

A neutral role records authorship: human, agent, system, or tool. A neutral part records meaning. Tool arguments must preserve JSON numbers and nesting without loss. The source's normalization bug rounded a large integer through floating point and produced valid JSON asking for the wrong thing. Syntactic validity was an unusually convincing disguise.

Calls retain their issued correlation IDs, even when those IDs look foreign to the next provider. Only a genuinely missing ID is synthesized, deterministically from stable event identity; multiple calls must remain distinct. Results refer back to those identities and retain their failure status. A result's function name can be recovered from its matching call rather than copied into permanent state just because one API needs it again.

Blob parts preserve references, not payload bytes: a local path, remote URI, or framework handle remains distinct. A request may eventually encode bytes as base64, but that encoding is not a

new historical fact. Merely reading a reference does not authorize opening a file or fetching a URI. An unsupported unredacted blob causes a local rendering error instead of silently disappearing.

The primary prompt is also generated at the rendering boundary, from context and configuration. It can be a constant at this stage. Keeping it out of stored context leaves later tool descriptions and capabilities derivable from their actual definitions rather than a growing collection of hand-maintained claims.

Translation may change shape, not authorship

The bridge targets Anthropic Messages, OpenAI Chat Completions, and Gemini GenerateContent. Their formats are boundary contracts, not alternative definitions of the agent's history. Provider behavior changes; the rules here describe the chapter's supported surfaces and conservative replay policy, not every model or future API.

A tool result remains tool-authored when Anthropic carries it inside a user message or Gemini places a function response in a user turn. OpenAI Chat Completions has a tool role. Merging several neutral entries into one wire message does not merge their authors or erase their original order in context.

Anthropic's relevant constraint is adjacency and block order, not simply alternating speakers. Results must follow the assistant message containing their calls, with result blocks before ordinary text in the carrying user message. Consider a reply asking for two tools, followed by one result, a human instruction, and the other result. Appending blocks as they arrived produces result, text,

result. The renderer must collect the results first, preserving their relative order, then place the remaining content. The disposable request gets reordered. History does not.

An interrupted call is harder. No recorded return means the execution outcome is unknown, not that execution failed or never started. The renderer first searches for a real correlated result, including a later one. Where pairing is required and none exists, this bridge supplies a deterministic wire-only error result: “No result was recorded; execution outcome unknown.” It uses the exact call ID and sits beside the other results before intervening human text. The error describes a missing record, not an observed tool failure.

A subsequent rendering uses a real result if one has since arrived. The placeholder never becomes a historical event. The renderer does not execute anything to complete the picture, borrow another call's result, or guess through duplicate and ambiguous IDs. Unrepresentable pairing fails locally before HTTP.

Gemini's function response needs an object envelope. Wrapping string content is not enough: already-object-valued content must retain the result's neutral failure flag too. Otherwise a convenient pass-through path turns a failed tool into an apparently successful response. On the way back, Gemini may report `STOP` while returning a function call, so pending work is determined from the parsed parts, not that label alone.

Empty text also has meaning. In the source's OpenAI failure, a token-limited response contained an empty string. Parsing discarded it; the next rendering emitted `null` for the bare assistant turn, and the provider rejected the request. A fake service had accepted it. Empty text, absent text, and an absent assistant turn are not interchangeable. An adapter that preserves elaborate tool metadata but drops an empty reply still loses conversation state.

Offline replay can expose these transformations without credentials or a network call. It shows the request that a particular context and configuration produce. It does not prove live provider acceptance. Conversely, one successful provider response does not prove that every boundary preserved meaning.

Reasoning is not ordinary assistant prose

The canonical model distinguishes readable thinking text from ordinary text and from opaque replay material. In this chapter, readable thinking is marked by `TextPart.Thinking`; opaque bytes have their own representation. That is semantic information, not decoration to recover later by searching for a heading in a string.

Opaque material may belong to a turn or to an individual call. The distinction became necessary when Gemini supplied call-bound thought signatures: an unattached bag of opaque parts could preserve the bytes while losing which call they accompanied. A signature therefore stays with its own call. Missing material cannot be repaired by borrowing a sibling's signature.

Every such artifact needs provenance: vendor, exact model, and API surface. The requested routing alias may differ from the actual producer named in the response, so both remain distinct. Render targets likewise need an explicitly resolved identity. A friendly alias that has not been resolved does not establish permission to replay a producer's bytes.

This bridge uses exact producer matching for opaque replay. It does not assume that a vendor-wide match is sufficient, nor that an HTTP success means a foreign signature was understood.

The source's September 2026 probes undermined its initial simple account of cross-model acceptance; some foreign material was accepted, some corrupted or missing material rejected, and some incompatible material could be dropped. Those observations do not establish a general compatibility promise.

The renderer therefore emits opaque material only to its issuing model through the corresponding surface. If a call requires replay material unavailable for the target, rendering fails locally rather than fabricating it or using a bypass marker. Equivalent responses from different providers can yield equivalent dialogue and usage while retaining different provenance; making every field equal would erase a fact.

Readable reasoning can remain in history without being resent as ordinary prose when opaque replay is the appropriate channel. Opaque content is transported, not obeyed. Its bytes do not gain instruction authority because they happen to decode into readable words.

Redaction changes the working set

The log owns evidence; context owns what is still useful to send. Redaction adds a new event describing a change to the latter. The original payload remains on disk. This is exclusion from future requests, not secure deletion.

Bill's reported experience with Gemini's SDK compaction explains why age alone is a poor selection policy. Compaction summarized the oldest portion of the session, including the task in the opening message. The agent remained fluent but lost what the work was for. A handoff document preserved continuity that the automatic summary had discarded.

The chapter instead gives content categories distinct redaction meanings. Replacing tool-result payloads with stubs retains what was asked. Removing calls as well as results loses more, while leaving surrounding prose. Removing text includes readable thinking text. Summarizing a whole span replaces its contents with compressed prose. These are different losses; a tool result can still contain indispensable evidence, so categories support a policy rather than settle it automatically.

Non-summary redactions filter entries individually. A summary replaces the entire span with one System entry at its starting sequence. The source records an implementation that used the same per-entry loop for both and inserted the summary into every entry. Compaction made the context larger. Summary replacement is a different operation, not another predicate in a filter.

Generated summary prose must be stored in the redaction event because replay cannot regenerate it deterministically. Ordinary stubs can be derived from what they replace. Blob stubs retain the original reference kind and locator, preserving where the bytes live without sending their payload. Context need not maintain a second, ever-growing registry of every redaction; the log already records them.

The same ownership explains the choice not to rely on provider-maintained conversation continuations. This agent needs local control over the next context after redaction and compaction. Resending a projection has a cost, but it leaves that control with the agent. Hosted conversation state is not inherently wrong; it serves a different ownership tradeoff.

Errors and usage must have one authority

An exchange failure and an unsuccessful tool result mean different things. A connection reset or HTTP rejection belongs to the exchange error path. A compiler that ran and found a type error produced a real tool result marked unsuccessful. Treating that result as a network problem invites retrying broken code while waiting for the connection to improve.

Diagnostics have a different authority again. A usage warning or a local rendering explanation must not become dialogue or duplicate a state-changing exchange failure. Parsers return neutral response and error events; History applies their defined semantics. Error text is an explanation, not a substitute for a structured decision about what the turn does next.

Usage follows the same single-record rule. The provider adapter returns normalized counts in the response event. Agent records that event, and History derives cumulative accounting. Engine measures the exchange but owns no independent mutable lifetime total to reconcile with replay. A response recorded twice or an Engine counter added again during reload would charge the same fact twice.

The shared categories are fresh input, cache writes, cache reads, and output. They do not overlap. Anthropic's reported input is treated as fresh input alongside separate cache categories. OpenAI's input total includes cached reads, so fresh input excludes them. Gemini's prompt total similarly includes cached content; output must include thought tokens where the response reports them separately. These are normalization rules for the supported response shapes, not a reason to guess when another shape appears.

Omitted optional cache fields can mean zero. Supplied negative, fractional, malformed, or inconsistent counts cannot. If cached input exceeds total input, subtracting would produce nonsense; independently valid output can remain known while affected input accounting becomes unknown. A missing response leaves no basis for inventing exact token usage, even when a remote system may have done billable work.

Unknown is not zero, and incomplete usage cannot honestly yield an exact cost. Each component needs its known or unknown status. Known contributions can still be retained, but adding a later well-formed response does not fill a hole in an earlier one. Raw provider usage remains available with a bounded diagnostic for investigation rather than being replaced by a plausible total.

An old log is still an input contract

Readable JSON does not make compatibility automatic. Malformed lines and unknown event, role, or part tags fail with a line-numbered error. A known event with optional fields absent is a different case: explicit defaults may supply meaning where the schema permits it. Silently mapping a new tag to zero merely hides an unsupported fact.

Stable wire names remain separate from incidental CLR names. New meanings need a schema-version policy: explicit support or rejection, and migrations that produce a new artifact rather than rewriting the original record. The same discipline applies to provenance enums: unknown names and invalid zero values cannot quietly select a vendor or surface. Model identifiers remain an open set compared for identity, not a growing enum of product names.

One canonical history now connects human input, model replies, redaction, accounting, and provider-specific requests. Chapter 1's simple exchange still works, but its record can also distinguish a requested action from a dispatched action and an observed result. The next chapter can build the tool turn on those distinctions without allowing a wire format to decide what actually happened.

Chapter 3: Six Tools, Ninety-Two Percent of an AI Coding Agent

A model that can describe a repair is useful. A model that can read the relevant file, make the repair, and run the tests becomes a different kind of application. The new capability comes from effects, not from a more elaborate conversation window.

The obvious temptation is to expose every operation the application can perform. The project's earlier tool-use records suggest a more selective starting point. Across 511 recorded sessions, the five most-used tools accounted for 91 percent of calls; thirteen accounted for 98 percent. Many carefully imagined tools barely appeared. Targeted editing was used far more often than whole-file writing.

Those figures describe that workload, not a universal distribution of software work. They are still a good reason to begin with a small, useful tool surface rather than a catalogue of possible features.

Declarations are part of the request

The model cannot call a tool merely because a method exists in the application. The request must advertise a name, a description, and an argument schema. Different providers spell that declaration differently, just as they spell conversational content differently. The tool's meaning belongs to Ensemble; the provider projection belongs at the protocol boundary.

A library Agent may have no tools. Its ordinary requests should then remain ordinary requests. The CLI installs the six initial operations as one application choice, rather than making every consumer inherit a shell and filesystem access.

In C#, a Tool is an owned object with a model-facing definition and a concrete operation. Its constructor registers it with the Agent. The implementation does not need an interface wrapper merely to connect that definition to its body. A small explicit dispatch over known operations is easier to follow than an extensibility framework built before an extension requires it.

There was an instructive testing trap here. A fake provider can volunteer a tool call whether or not the client advertised any tools. The resulting loop looks correct locally and does nothing against a real provider. Exercising the loop therefore does not establish that the request contains the declaration. A useful fake reproduces enough of the external contract to expose that distinction, and tests inspect the outgoing request as well as the returned result.

The assistant proposes; the application executes

A response can contain text followed by one or more tool calls. The application records that response before performing the effects. Each call has an identity, a name, and input. Execution produces a result tied to the call's identity, and that result becomes input to another model request.

The call identity is essential when several operations are outstanding. A result named `read_file` cannot distinguish two reads of different files. Position is no better once another component observes or serializes the events. If a provider omits

an identity, Ensemble assigns a deterministic distinct one while capturing the call, before logging or dispatch. Inventing it later during serialization would leave the execution path and the wire using different facts.

The loop continues until the model returns without asking for tools, or reaches a bounded round limit. A failed command is usually information for the next model round, not a reason to pretend the whole conversation succeeded or to crash the application. Unknown names and invalid arguments likewise become explicit tool failures without performing the requested effect.

Protocol details still matter. Anthropic's result-bearing messages require tool results before accompanying material. That ordering becomes especially important once hints can arrive during a tool call. The canonical result and its identity stay the same; the provider renderer owns the legal arrangement on the wire.

A short explanation before a tool batch gives the human a chance to understand the proposed action. Ensemble can require that explanation through explicit configuration. It does not infer the policy from a vendor's name. When enforcement is enabled, a calls-only response receives explanatory failures rather than execution. The next round can explain and try again. This is an observable application rule, not a claim that a model's private reasoning has been exposed.

Six operations, useful edges

The initial tools cover reading, writing, targeted editing, listing, searching, and running a command. Their ordinary behavior is familiar. Their edges determine whether the assistant can use them safely and economically.

A read should expose enough location information to support a later edit. A search should return paths and matching lines rather than a decorative summary that requires another discovery pass. A directory listing should give a predictable view of what is present. The model pays for ambiguity with additional calls, and the user pays for those calls.

Writing and editing deserve different operations. Creating a file, appending to one, and replacing its contents have different consequences; they should not collapse into an accidental overwrite. Targeted editing takes an exact old anchor and a replacement. Zero matches means the model's view is stale or wrong. Multiple matches mean the request is ambiguous. Neither is permission to choose a location and hope.

This explains the measured preference for editing over writing. Replacing a small unique passage preserves unrelated work and avoids regenerating an entire file to change one line. Refusal is useful information: reread the relevant region and form a better anchor. A guessed line number would merely disguise uncertainty as precision.

The command runner introduces a broader effect boundary. Output and exit status both matter. A nonzero process exit is a normal tool result: the command ran, and its answer may be that the tests failed. Marking that as a tool-call error would invite the model to repair a call that was valid. The result preserves diagnostics and exit status, so successful execution of the tool cannot be mistaken for successful execution of the tests.

Working-directory changes apply to the particular invocation. A command must not quietly change the application's global directory and redirect later file tools. Fresh calls do not inherit an interactive shell's incidental state. A long-lived interactive process will eventually need its own explicit identity; it should not emerge accidentally from a global shell session.

The early filesystem restrictions provide a bounded workspace. The fuller sandbox chapter will examine canonical paths, symlinks, and the difference between filesystem policy and an operating-system security boundary. It would be misleading to promise all of those properties merely because these six tools accept a root directory.

Teach the errors

The model's next action depends on what the tool says happened. A useful refusal identifies the violated condition without making up an effect. A missing file is different from a refused overwrite. A nonunique edit anchor is different from a malformed argument. An unknown tool is different from a command that ran and failed.

The same applies to malformed model output. The application should not execute a fragment just because it resembles a useful operation. Tool names, argument shapes, and call identities cross a trust boundary. Parsing them into a plausible object is the beginning of validation, not the end.

There is no benefit in adding elaborate recovery machinery before these simple distinctions work. A direct loop that reports a failure honestly gives the next model request a chance to repair it. A supposedly helpful wrapper that converts every exception into a vague success makes the next request less informed than the last.

A new kind of silence

The six tools make the assistant capable of useful work. They also expose a limitation in the loop: control returns when the tool returns.

That is harmless for a short file read. It is much less harmless for a test suite, an interactive debugger, or an external tool that stops answering. The model can be perfectly capable of deciding what to do next and still have no opportunity to decide anything.

The next chapter gives work a lifetime of its own, so the conversation does not become its hostage.

Implementation details: codebook Chapter 3.

Chapter 4: Jobs, or Why a Tool Call Is a Process You Supervise

The failure that motivated this chapter was a screenshot. The tool stopped returning, the agent remained apparently busy, and there was no way to recover the conversation. Bill killed the process, taking unrelated work in the agent tree with it.

That detail matters. A repair limited to shell commands would not have fixed the screenshot. The problem was the shape of dispatch: the application handed control to a tool and waited for the tool to give it back.

A timeout seems like the obvious answer until the tool is a healthy build, a debugger, or a server intended to stay alive. Silence cannot distinguish those from a hang. The more useful question is how long the caller wants to wait before looking again.

Give the operation a place to live

An asynchronous handler already continues independently of its caller. Abandoning the wait does not necessarily stop it; it may simply discard the only convenient route to its eventual result. The job system keeps that route.

Every ordinary tool call passes through one dispatch path. The dispatcher allocates a Job, gives it a handle, records the call, and only then releases its worker. A logging failure before that release

can refuse the operation without an unrecorded effect escaping into the world. The Job owns the lifecycle and retained output; the caller owns an observation of it.

Fast work still returns a normal completed result. Slow work returns a running report and a handle. The caller can inspect the same operation later rather than launch another copy and risk duplicated effects. A running report is an answer to the observation, not an answer from the unfinished tool.

The distinction removes a whole class of alarming error messages. Without a handle, an abandoned call needs a warning that it may still complete and should not be blindly repeated. With a handle, the application can say where the work is and how to return to it.

Handles are allocated across an Ensemble so its Agents do not accidentally reuse them. Output belongs in each Agent's data directory. A convenient CLI directory is not a global location that every embedded application must share.

Put supervision above the tools

A file read, a directory listing, a shell command, and a tool supplied by an application all need the same observation rules. Otherwise responsiveness depends on knowing in advance which tools might block. External tools make that prediction especially unreliable.

The job boundary therefore lives at dispatch, not inside six individually improved handlers. A new ordinary tool inherits supervision by entering through that boundary. Its author does not get to opt the whole conversation back into an indefinite wait by choosing an unfamiliar name.

The controls that supervise jobs are the deliberate exception. Waiting for a job must not recursively create another ordinary job that then needs supervision itself. Input, waiting, and termination operate on the existing handle. A one-shot observation setting changes how a subsequent call is watched; it is not a new execution subsystem.

This separation also makes ownership intelligible. The Agent owns Jobs, Jobs owns each Job, and the work's resources remain reachable through those relationships until cleanup has finished. The observation may be brief. The ownership cannot be.

Keep the full result, bound the conversation

A megabyte of test output is a useful artifact and usually a terrible model message. The framework needs both facts to be true.

The full tool output is retained on disk. Each observation returns a bounded preview, the job's status, and a locator for the rest. Later waits return newly available output rather than replaying the entire transcript on every poll. The model can request a larger excerpt or inspect the artifact when the next decision actually requires it.

Truncation must be visible. A shortened preview that looks complete can make a passing build appear to have no final status, or hide the one diagnostic that matters. Status and exit information therefore remain explicit instead of being inferred from whichever text happened to fit.

Durable output also improves conversation reduction. When a large result no longer belongs in working context, a compact reference can still lead back to the actual evidence. Without

retained output, the only recovery instruction would be to rerun the effect. That is wasteful for a read and potentially dangerous for a mutation.

These jobs are not a durable distributed queue. Their output can survive for inspection, but their execution lifetime belongs to the running application. Application exit requires shutdown, not a fiction that every in-process task will continue elsewhere.

Observation is a call policy

Early watchdog designs tried to infer how long each tool could legitimately block. A table of tool names missed unfamiliar tools. Scanning descriptions for timeout hints added another unreliable source of truth. Requiring every tool to declare a blocking limit repaired the table while preserving the wrong question.

A caller may want to check a build after a fraction of a second, wait longer once it is clearly running, or wake when a debugger prints a recognizable prompt. Those are choices about an observation. The tool's identity does not determine them.

Ensemble lets the call select its delay, output cap, and optional pattern. Expiration wakes the observer; it does not kill the operation. Pattern matching is another reason to look again, not evidence that the process has finished.

A one-shot override applies to the next call and reports where it was consumed. That last part matters. A model can prepare a generous output allowance for a file read, glance at a directory first, and accidentally spend the allowance there. Silently retaining or guessing the intended target would create more hidden state. Naming the consuming call makes the mistake local and repairable.

A completed job remains observable too. Completion does not erase its handle just because the first caller was absent when it happened.

A terminal is a conversation with a process

An interactive debugger needs more than a command string and a future exit code. It prints a prompt, waits for input, and produces another prompt. A pseudo-terminal supplies that interaction, and the Job supplies its identity.

The shell tool therefore runs with terminal semantics. Standard output and error are merged in the terminal stream, while exit status remains a separate fact. The process can receive additional input through its handle. The observer can wait for a prompt without treating the prompt as termination.

A fresh shell invocation starts fresh. Changing directory or exporting a variable inside one call does not silently alter later calls. State persists when an explicitly retained interactive process owns it, not because the application happens to reuse a global shell. A working-directory override belongs to the invocation and is reported as such.

That model makes a debugger, a test runner, and a development server variations of the same supervised work rather than three unrelated tool frameworks.

Cancellation must tell the truth

Ending a wait leaves the operation alive. Explicit termination asks the owned work to stop and records why. Application shutdown ends all remaining owned work before the Agent's relationships disappear.

The mechanism depends on what is running. A pseudo-terminal process group can be terminated and drained. Arbitrary managed code cannot safely be forcibly aborted by .NET. Its handler receives cancellation and must cooperate. A handler that ignores cancellation can delay shutdown; changing its recorded status does not make its thread or effects vanish.

This limitation is not a reason to obscure the boundary. It is a reason to document the contract of application-supplied handlers and keep their resources attached until actual cleanup is complete. The parent paths used for logging and final reports must still exist while workers are finishing.

Supervision gives the agent a way back to unfinished work, a bounded view of large results, and an explicit way to end what it owns. It does not yet guarantee that the agent can hear a new instruction while watching a job. If the conversation loop itself is waiting, the user can still be talking to an unread inbox.

That is the next ownership problem.

Implementation details: codebook Chapter 4.

Chapter 5: Two Seams and a Loop

The test is running in the wrong directory. The user types a correction. The application accepts the text, and nothing happens.

Background execution has solved only half the problem. A tool can run independently while the conversation loop remains parked waiting for its report. The input is present, but the owner that must interpret it is asleep.

The repair gives conversation state one owner that stays available. Transport and tool workers bring facts back to that owner instead of borrowing its history and changing it themselves.

One owner orders the conversation

Each Agent has an actor: a loop with a mailbox and the authority to advance that Agent's conversation. Prompts, hints, control requests, transport outcomes, and tool reports enter through the mailbox. The actor applies their facts, records the resulting history, and decides what may run next.

This is not a second framework hidden inside the application. The C# implementation uses async operations to perform waits outside the conversation owner. Its lasting scheduling fields are declared with the Agent in Common. A worker does not acquire permission to edit History merely because it has an object reference capable of reaching it.

That distinction is more useful than counting threads. The application may need transport workers, process readers, tool handlers, and observers. The important count is the number of authorities ordering one conversation: one.

Admission order also needs a single meaning. Separate independently selected channels for prompts, hints, and completions could invent a new ordering after the facts had already arrived. The mailbox provides an order in which this Agent accepts work. It does not pretend to discover a universal chronology among simultaneous events elsewhere.

Freeze before sending

To start a model request, the actor records the input, renders a frozen request, records its request boundary, and releases the HTTP operation. At most one model request is active for that Agent.

The transport receives the frozen request, not a live History object from which it can assemble a slightly later conversation. It returns immutable response facts without assigning their final place in the log. The actor supplies that order when it accepts the result.

A hint arriving after the request was frozen cannot retroactively change its bytes. It belongs in the next request. Recording that boundary makes the distinction reconstructible rather than dependent on when a background thread happened to read a mutable collection.

Tool execution has a similar admission prefix. The actor validates call identity and records the call before a handler is released. The subsequent wait for a report runs elsewhere and posts its outcome back. Long observation does not hold the mailbox hostage.

A report is recorded before the next model request consumes it. A directly invoked host tool has a real audit outcome too, but should not invent an assistant tool call that the model never made. Both cases preserve evidence; they have different authorship.

Two public seams

An application needs to send work into the Agent and observe what comes out. Those are separate operations with different reliability requirements.

The inbound seam admits prompts and controls. A submitted request receives its own completion task. Concurrent submissions queue separate turns rather than racing to read a shared last-answer field. The actor settles each admitted request exactly once, and asynchronous continuations prevent an application callback from taking over the actor while the result is delivered.

The outbound seam carries progress through bounded consuming channels, one on the Agent and a separate merged channel on Ensemble. Two readers of the same channel compete for values; this is not broadcast subscription. A consumer may also discard nonmatching values while waiting for a particular observation. Later GUI fan-out needs one reader and a separate projection for its viewers. Observation can lose progress when a consumer is slow. Letting a browser stall the actor because its output queue is full would make presentation part of the application's correctness path.

The reliable completion task therefore does not depend on seeing every progress event, noticing an arbitrary Idle transition, or guessing that the last text chunk was final. A caller waiting for an answer and a window watching tokens are not doing the same job.

Cancellation follows that distinction. A caller can detach from its wait without cancelling the admitted turn. Ending the turn requires an explicit interrupt. Closing a tab should not silently become authority to destroy work that another consumer still cares about.

With durable logging enabled, recording precedes publication and completion. An answer cannot be reported as durably successful if the append that establishes it failed.

Make the library usable without its front end

An earlier architecture exposed the cost of letting a framework import its own GUI. The supposed library depended on the application that was meant to consume it. A new front end could not simply borrow the agent; it inherited composition decisions it did not own.

Here, the application constructs the root, configuration, and Agents. A CLI translates terminal input into submissions and controls. A GUI does the same for browser messages. Neither owns a private copy of the conversational loop.

This separation matters before there are multiple user interfaces. It keeps stdin, window state, and application exit out of the provider and history code. It also gives tests and small external programs a real public seam instead of a privileged back door available only inside the repository.

Input capability belongs to the selected model and provider, not to what the front end can display. Successful download or decoding does not establish that a particular model accepts an image, audio, or video input. The early conservative defaults admit images and refuse other media unless configured support

says otherwise. Unsupported input is refused before its history event is recorded. A browser's ability to play a video is a separate presentation fact.

Hear the correction

A hint is a useful test of the whole design because receipt alone proves very little.

During HTTP, the actor retains the hint for the next request. During foreground tool observation, it ends that observation without killing the independent Job, obtains the actual running or completed report, and returns control to the model with the hint visible.

A response may have requested several tools. If a hint or control boundary prevents the undispatched tail from running, those calls still need exact-ID results saying they were not executed. Leaving them dangling would corrupt the next provider request. Executing them later after the model has changed its plan would be worse.

An idle hint does not necessarily start a fresh turn. It can remain available until an actual trigger arrives. That prevents a seemingly small control message from becoming an unrequested paid generation.

Pause is different again. It prevents new model and tool scheduling while the actor continues to receive input and process completions. Resume allows scheduling to continue. Freezing the mailbox would make pause unable to hear its own release.

Immediate interrupt cancels the active transport and foreground observation, then ends the current turn. It does not automatically kill independent Jobs. An after-tool interrupt preserves the already admitted response or dispatched-tool

boundary and refuses subsequent effects. In both cases, late outcomes must be collected without letting them dispatch stale calls or settle a newer request.

These distinctions take more care than one global cancellation token, but they correspond to operations the user actually means: stop showing me this, stop this turn, stop starting new work, or terminate that process.

Bound the loop without mutilating the answer

A tool-round limit bounds how much work one human turn can generate. It counts batches, not individual calls. After the last permitted batch, the model still gets the results and can answer. If it asks for another batch, the application refuses those effects and pairs the calls with explicit results.

Editing the model's returned content to conceal its extra calls would destroy the record and can invalidate signed content. The limit belongs at the execution boundary, where the application has authority to refuse an effect, not at a fictional rewrite of what the model said.

The same principle governs shutdown. Close admission, settle already admitted requests, cancel and join the transport and observations, stop owned Jobs, and only then detach the model. History and parent paths remain available during cleanup. Repeated shutdown joins the same lifetime rather than starting a competing cleanup operation.

A managed callback that ignores cancellation can still delay completion. The actor design improves ownership and responsiveness; it does not give .NET a safe arbitrary thread-abort operation.

The result is a library whose conversation can be watched, addressed, paused, and stopped without confusing those actions. With that separation established, streaming can finally be treated as what it is: a better way to observe an answer arriving.

Implementation details: codebook Chapter 5.

Chapter 6: Streaming, or the Same Answer in Pieces

An HTTP response is already a stream of bytes. Waiting for its entire body before showing anything is a client decision. Removing that wait improves the human's ability to follow the Agent, but it also creates two kinds of truth: what has arrived so far, and what the provider has actually completed. The display can live with the first. Conversation history and tool execution need the second.

One answer, two delivery paths

The Engine accepts the open HTTP response after headers arrive. For plain JSON, it reads the body and invokes the established completed-response decoder. For server-sent events, it recognizes frames, emits display chunks, and accumulates the provider's native response. Only after successful completion does it send that accumulated response through the same decoder.

That shared finish line matters more than the callback. Separate JSON and streaming decoders would gradually disagree about exact argument values, nullable usage, signatures, and opaque replay material. Streaming changes when content becomes visible without creating a second definition of a valid answer.

Plain delivery invokes the chunk callback zero times. There is no synthetic one-chunk "stream." It still produces authoritative final parts, so a display must create a card from a final even if no delta preceded it. When deltas did arrive, the final replaces their provisional content rather than being appended to it. Otherwise the most successful response would display twice.

Common declares the value-like chunk and parsed-response vocabulary and the required partial parsing operation. Engine supplies the implementation in the same C# 14/.NET 10 assembly. The invocation's callback carries transient parser output; it is not a new service or an alternate owner of conversation state. Accumulators are per-parse scratch state, released when decoding finishes.

Progress travels through the owner

The HTTP worker posts chunks to the Agent's existing mailbox, then posts an immutable completion. It cannot append History, dispatch tools, or finish the turn. Controls and streaming progress therefore meet at the same actor rather than competing through separate loops.

The actor publishes deltas to bounded, lossy observation queues. A slow terminal may miss a fragment; it cannot stall the Agent. Final events are different: the actor records them before publishing their finalized parts. Their notifications can still be lost; the stronger promise is the record, not guaranteed delivery. An observer that checks History upon receiving a final can already find the fact there. Deltas never become extra dialogue or additional token accounting.

A reliable request task establishes completion. Neither a quiet screen nor a final-looking fragment can do that, and losing an observation does not change the request's outcome. This

separation also explains the terminal arrangement: live text, thinking, and tool fragments are a stderr preview, while the reliable completed answer goes once to stdout. Redirecting stdout must yield the whole answer even when a display queue dropped progress. JSON observation output flushes each delivered record so streaming is visible through a pipe rather than merely happening inside the process.

Identity survives rearrangement

A chunk needs to identify the part it extends. Allocating a new ID for every fragment would describe forty unrelated objects instead of one developing paragraph. Numbering the final array is also too late: a tool fragment may arrive before text that the canonical decoder places ahead of it.

The parser that chooses an ID therefore returns it again alongside the finished parts. IDs are response-local, nonnegative, and distinct across parts; zero is valid. The actor supplies the recorded request sequence as the response identity. Across several Agents, the display key combines Agent, response, and part. A provider's tool-call ID remains a separate correlation identity, not a convenient substitute for a display key.

Each adapter translates its own wire semantics. Anthropic supplies block indices, including gaps. OpenAI tool indices need mapping to parser-local identities, while text and thinking acquire identities on first appearance. Gemini has no equivalent block index, so continuation depends on content kind, explicit boundaries, and signature closure. Empty signed text can matter to replay even though it paints no characters.

The final mapping preserves identity when canonical ordering differs from arrival order. Arrival order itself remains important for the explanation policy: substantive text that arrives after the

first tool fragment cannot retroactively explain that call. The parser records that fact before final decoding rearranges anything.

A visible fragment cannot authorize an effect

SSE framing is modest but unforgiving. Blank lines delimit frames; multiple data lines form one payload. Comments and CRLF must work, as must a pending frame at EOF. A frame is not a part: either can span the other. Protocol completion still has to be present. A connection ending after readable text is not sufficient evidence of success.

Vendor completion rules remain local to their adapters. Anthropic cannot succeed with an open block. OpenAI may send usage after the finish reason, so the parser accepts that accounting without accepting new choice content. Gemini's signatures must survive on the kinds of parts they actually sign. None of these rules justifies dispatching a partly received tool name or argument object.

The entire response must finish successfully and the arguments must decode before tool admission is possible. A malformed frame, provider error, missing terminator, or cancellation can leave a persuasive preview on screen without yielding a successful recorded response. Expected protocol failures become credential-safe errors; cancellation returns through the actor. Completed calls refused by the explanation policy still receive paired results, but produce no effect.

Exact JSON values survive this path. In particular, a large integer must not quietly pass through floating-point storage and emerge rounded. Comparing decoded argument values is more

meaningful than demanding that streamed whitespace match canonical JSON formatting.

Three useful records

History preserves completed conversation facts. Observers offer timely, expendable progress. The optional API trace preserves sanitized request and received-frame payloads for diagnosis. These serve different readers and should not be mistaken for interchangeable logs.

The root serializes trace writes to a caller-owned writer, reached through the Engine's actual Agent and Ensemble relationships. Redaction removes the configured key and its escaped representation, but does not make the trace public-safe: prompts, files, and signatures may remain. It is a payload trace, not a byte-for-byte HTTP capture or a general secret detector.

Streaming selection is equally restrained. A known model must allow streaming and the caller must not disable it. Unknown models fall back to plain delivery; losing immediacy preserves the answer. Unsupported media still fails before HTTP because silently losing an attachment would change the answer's inputs. A capability-table entry can select a known parser. It cannot teach that parser a new protocol.

Source: Chapter 6.

Chapter 7: Everything Is an Artifact

Text, thinking, tool arguments, and results compete poorly for one terminal cursor. The browser gives each a stable place without acquiring authority over the Agent. Closing a tab must not cancel a turn or kill a job. A button sends the same public input that a non-GUI caller would send; the existing actor still schedules work and records what happened.

The GUI is an optional .NET 10 application using ASP.NET Core. Its composition root constructs the runtime and host. Its own Common declarations and server implementations remain separate from the core library, which knows nothing about ASP.NET. Connection queues and partial display text live for the hosting operation; they are derived state, not another transcript owner.

Reconnection needs facts, not memories of notifications

The browser has two content sources. Finalized responses, human input, tool admission and results, and terminal facts come from History. Partial text for an unfinished response comes from the host's accumulated observations. Treating both as a recent-notification list would make a lost notification indistinguishable from something that never happened.

History supplies an atomic snapshot of events and reduced context. Its append boundary validates, sequences, writes, and flushes before publishing the new state. The following observation may be dropped; the recorded event remains available. The server can therefore repair finalized content after a quiet interval or reconnect without inventing replacement deltas.

Connection registration, replay construction, and queueing share a short handoff lock. A new client gets replay before becoming eligible for subsequent frames. Network sends happen outside that lock, with a separate bounded output queue and writer for each socket. A browser that falls too far behind is disconnected rather than allowed to block every other client or the actor. Its recovery path is the ordinary replay path.

Ordering needs more care than "take the latest snapshot." By the time an old delta reaches the host, History may already contain the final response. Publishing that newest snapshot first would erase genuine live progress. The live path projects through the observation's recorded sequence; full catch-up happens on idle polls and connection establishment. Conversely, a reconnect that already received the final must not revive the part when an old queued delta arrives.

This restores a view of the same live Agent. Restoring an application after process restart requires the persistence work of a later chapter.

The card has one identity

Every displayed message is an artifact: a stable card whose rendering varies by role and kind. A single card map handles human text, assistant text, thinking, planned calls, and results. Separate widgets do not require separate conversation models.

For model parts, Chapter 6's parser identities now survive into recorded response metadata. The response identity is the actual request sequence; part IDs follow canonical part order while retaining the parser's original values. The browser key combines Agent, response, and part. Human messages, results, and older records without parser metadata use a distinct deterministic event-and-position namespace.

That separation avoids collisions without a global display counter. A final replaces a partial card; replay reconstructs the same card; a repeated final is harmless. Tool-call IDs keep their provider correlation meaning, and opaque signatures are preserved rather than rewritten to suit a widget.

The page and its static assets come from the loopback GUI host. Its `/ws` connection carries replay, live display, and controls. Vendor SSE never reaches the browser: Engine has already translated it into the shared vocabulary. A subscribe request is idempotent, not permission to create another Agent or replay execution. The host's history frontier is internal bookkeeping, not a browser-supplied authority over which facts exist.

A plan is not a started job

Tool cards need three distinct moments. The model requested a call; the application admitted execution; an observation returned. `ToolCalled` establishes the first. `ToolStarted` records the second, before effects are allowed to begin. If that recording fails, execution cannot start.

A refused call therefore has a request and a paired failure result, but never appears dispatched. An admitted command that later cannot start is different: admission happened, and its

outcome failed. Keeping those facts separate makes the UI useful for understanding both policy refusals and operational errors.

Even “finished” needs a precise scope. A foreground tool observation may return a running-job handle while the underlying job continues. The result card retains that handle and its locator rather than claiming the process exited. Likewise, turn completion comes from an explicit terminal fact, not a momentary Idle state between tool rounds. The answer is derived from the recorded response rather than stored in a second authoritative answer field.

Compact presentation, complete content

Collapsing thinking or tool details saves space without discarding their contents. Scrollable code and readable search results preserve the rest of the evidence. A successful exact-replacement edit can show the applied replacement from its admitted arguments, but that is not a whole-file diff.

Markdown rendering constructs safe DOM nodes. Provider and tool text does not become executable HTML, and terminal escapes do not become browser controls. A source file containing HTML should appear as source, even when its contents would make an excellent attack on a careless transcript renderer.

The full backing text also feeds each artifact’s speaker button. Reading a card means reading the whole human message, assistant answer, thought, tool call, or result—not just its collapsed preview. Opaque signatures are never speech. Selecting another card replaces the previous speech; selecting the active card again stops it.

Automatic narration is a separate opt-in behavior. It reads new assistant text and thinking as complete clauses arrive, flushes the last remainder, and announces concise tool-call summaries. It does not automatically read tool results, human input, or replayed history. Reconnection should not trigger a recital of everything the human already heard.

Speech and typing steer scheduling

The speech queue counts as busy before audio actually starts. Otherwise the gap between enqueueing and the browser beginning playback becomes a scheduling race. Both speech completion and speech failure settle the current item; callbacks from cancelled items are ignored by identity. The historical Chrome visibility workaround—an empty, silent utterance when the page becomes visible—stays outside that queue so it cannot manufacture a speech pause.

Typing or queued speech creates a local pause cause. Its false-to-true transition sends pause; its true-to-false transition sends unpause. The actor stops starting new model requests and tools while continuing to accept controls and worker completions. An in-flight request may finish and be recorded, with its planned tools waiting for unpause.

These controls are edge-triggered because pause is Agent-wide. Another tab or caller may explicitly unpause while this tab still contains text. Unchanged local typing must not reassert a permanent veto on every state update. A later local transition can issue another request, but the server's effective state remains authoritative.

Loopback binding and origin checks protect this local control surface from casual cross-origin access. They do not sandbox the Agent's tools. The browser makes work easier to inspect and

steer; it does not reduce the consequences of the commands the Agent is authorized to run.

Source: Chapter 7.

Chapter 8: Build Your Dream GUI

A settings control makes a promise about the running system. If it says the font is larger, existing transcript text should grow. If it says the next turn permits three tool batches, the Agent should capture three when that turn begins. If saving fails, the page should not display a preference that only briefly existed in JavaScript.

The GUI keeps this promise by separating three lifetimes. The browser displays the last acknowledged snapshot. The server owns presentation preferences and their persistence. The Agent owns its next-turn tool budget, which an active turn captures and then keeps. None needs another scheduler.

Comfort without a second application inside the page

The layout remains deliberately small: one transcript, a prompt area, wrapping controls, and a native expandable Settings panel. It uses ordinary document flow rather than fixed overlays or a collection of nested scrolling panes. Code can scroll horizontally; long messages remain in the transcript. Theme and font changes apply to content already present as well as to new messages.

Light, dark, and system themes share CSS variables, with system theme following the operating system's preference. Native controls, associated labels, visible focus, and narrow-width

behavior do useful work without requiring a custom widget framework. A geometry assertion can reveal overflow; it cannot establish that every reader will find the page comfortable.

Narration and per-artifact speech remain separate. Disabling automatic narration does not remove a card's speaker button. Changing speech rate affects the next utterance constructed, not sound already playing. Turning off pause-while-speaking removes the speech-related pause cause without cancelling speech. Genuine typing remains an independent cause, so clearing one reason to pause need not clear them all.

One acknowledged value

The server holds a value-like preferences record for theme, narration, speech rate, font size, and speech-related pausing. A value snapshot needs no parent links or destruction machinery. The tool budget is deliberately absent from that record: the server reads it from the actual Agent when constructing a complete wire or disk snapshot.

A browser sends a patch, not a new authoritative settings object. Under the host's short settings lock, the server merges the patch with current values, validates it, and writes the complete candidate to a unique temporary file beside the destination. Only after replacing the destination does it update live preferences and Agent policy, then queue the same full snapshot to every connected client. Network sends remain outside the lock.

This order makes the acknowledgment meaningful. A write or replacement failure leaves the accepted in-memory values and prior good file unchanged, reports an error to the requester, and cleans up the owned temporary file. The page reverts its controls

to the acknowledged snapshot rather than retaining an optimistic fiction. Two tabs converge on the server's published state instead of maintaining rival local-storage copies.

The default file lives at `$HOME/.ensemble/settings.json`; an explicit application setting can select another path. Same-directory replacement avoids exposing half-written JSON. It does not promise power-loss durability or provide a backup archive. These are preferences, not credentials or conversation events.

Reading a bad file is different from failing to save

Disk loading and incoming patches use the same validation policy. Invalid fields retain their prior values; loading starts from defaults. Types stay meaningful: a JSON Boolean is not interchangeable with a string or an integer. Valid numeric speech rates and font sizes are clamped to supported ranges; a tool budget must be a nonnegative integer. Unknown fields do not become accidental new settings.

A missing, unreadable, or malformed file falls back to defaults without rewriting or deleting the original. This is an intentional change from the older edition's fail-startup policy. Broken presentation preferences should not lock the human out of the interface needed to recover. Keeping the file untouched preserves an opportunity to inspect it, although a later successful settings edit may replace it.

Read fallback and save failure therefore have different consequences. Startup can proceed with defaults when no usable preference snapshot can be loaded. An unsuccessful update

cannot claim that a newly requested value was accepted. Both rules keep the page honest while avoiding unnecessary downtime.

A budget belongs to a turn

The Agent's runtime policy is synchronized through its ingress lock. The GUI can read or update the next-turn value without examining actor-private state. The constructor configuration seeds this policy; it does not remain a competing mutable settings object.

Zero means automatic, resolving to 200 tool batches when a turn begins. It stays zero on disk, in acknowledgments, and in the control. Only the captured effective limit becomes 200. A queued prompt captures the policy when it actually starts, so a settings change can affect queued work without changing a turn already in progress. A hint continuing that turn does not mint a fresh budget.

The earlier library constructor's negative unlimited setting remains supported, but the new runtime setter and GUI accept only nonnegative values. Preserving an older API contract does not require exposing that convention as a new preference.

A batch is one model response containing tool calls, regardless of how many calls it contains. After the last admitted batch, the model still needs to consume the results and answer. The Engine adds a request-only instruction to finish without more tools. That instruction does not enter History or modify signed replay content. If the model nevertheless requests another batch, each call receives a refusal before any effect begins.

The budget thus has both an explanatory side and an enforcement side. The request tells the model how to finish gracefully; the actor prevents further execution. A settings panel

that only changes prompt wording would offer a suggestion where it had promised a limit.

Source: Chapter 8.

Chapter 9 — Skills

A tool needs both permission and an explanation. Registering a file writer gives the runtime a handler; it does not tell the model when to use it. A paragraph saying “write the file” teaches an intention; it does not grant execution. A skill joins these two facts: a rendered manual and a named set of capabilities, activated together and recorded as such.

The primary manual establishes the Agent’s working rules and tells it which additional manuals it may request. Supporting dependencies load before the manual that needs them. This keeps the initial instruction set manageable without reducing later capabilities to unexplained function signatures.

The record says what was taught

Each Agent owns a concrete Skills manager. Common declares its relationships, persistent fields, immutable skill values, and required partial operations. The functional Skills files implement catalog parsing, resolution, variable substitution, and activation; Agent implements the attachment relationship. No Go-style package interface or runtime adapter is needed to connect these parts.

The manager keeps catalog definitions and registered variables. History owns the activation facts. Active manuals and grants are derived from skill parts in reduced Context rather than

duplicated in a mutable enabled-tools set. Engine renders from those same facts, and Jobs checks them before reserving work or causing effects.

That common source prevents two unpleasant states: a model being shown a tool that dispatch considers disabled, and dispatch allowing a tool whose manual never entered the conversation. Registered handlers remain the Agent's catalog, not automatic permission in a skill-configured Agent. Library callers that do not configure skills retain the earlier explicit-installation behavior.

A tool granted by several manuals still has one effective permission. Its provenance is the earliest recorded activation that grants it, including that activation's sequence. The answer to "why can this run?" is therefore a conversation fact, not whichever dictionary happened to be inspected last.

A small catalog, a complete activation

The host supplies one skill directory. The manager recursively reads `SKILL.md` files in sorted order and rejects duplicate names; there is no hidden home-directory fallback or precedence contest between roots. The front matter deliberately supports a small documented subset rather than general YAML. Predictable errors are more valuable here than accepting an ambiguous configuration language by accident.

Discovery is narrower than "every file found." Only active manuals' explicit offers make loadable skills available. A dependency is not independently requestable merely because it sits in the same directory. Loading resolves the entire dependency graph, detects cycles, checks types and handler names, and refuses the operation if any required piece is invalid.

Validation completes before mutation. Newly needed dependencies and the requested manual are rendered, then recorded in one dependency-first activation event. Publication succeeds as a whole or not at all. A repeated load adds no duplicate manual. A broken dependency must not leave half its closure active.

Construction applies the same discipline to the primary. If activation fails after the manager attaches and installs its controls, those additions are removed and the relationship is detached. Destruction clears both sides of the relationship without destroying the parent. These are ordinary ownership rules doing useful work, not a special plugin framework.

Render once means remember once

Variables are substituted when a manual activates. The resulting text is the remembered manual. Updating a registered variable later affects future loads, not what the primary or an already loaded manual said. Editing or deleting the source Markdown likewise cannot rewrite a recorded activation during replay.

The primary is rendered first and stays fixed. Later manuals enlarge the complete instruction block without mutating that first body. This matters for built-in substitutions as well as custom values: a rendered tool list is a snapshot, not a live getter hidden inside prose.

Variable values come from the host. Core does not inspect the environment. Substitution is deliberately simple: names match case-sensitively, unknown references remain literal, and inserted text is not scanned again as another template. Neither recursive expansion nor shell syntax is smuggled into an instructional

Markdown file. The host still has to choose supplied values carefully; a value intentionally placed in a manual will reach the model.

A manual belongs in instruction context

A successful load result reports that loading succeeded. It is not the delivery mechanism for the manual itself. Anthropic receives manuals in its top-level system field, OpenAI Chat Completions in the initial system message, and Gemini in `systemInstruction.parts`. The primary comes first, followed by other active rendered bodies, each once.

Skill entries are excluded from ordinary dialogue projection. Burying the manual only inside tool-result prose would give it the wrong role and make its survival depend on ordinary result retention. Clearing a verbose load result must not erase the manual or revoke its grant. Nor should signed provider content be rewritten to carry new instructions.

The actor captures an immutable Context before rendering a request. Declarations use that snapshot's grants, rather than consulting a live set halfway through serialization. Dispatch checks recorded grants again before execution. A model that guesses a disabled tool name receives a paired error using the actual call ID; it does not create a job or file.

The local skill controls run synchronously at the actor's admission point, recording their call and result without an independent Job. The next model request sees the new instructions and capabilities. Skill loading has not invented another scheduler.

Unloading is intentionally unfinished here

This chapter marks a non-primary manual pending removal. Its instructions and tools remain active. The primary cannot be unloaded, and repeated pending requests do not add redundant state.

That behavior avoids claiming to solve a harder context problem early: abruptly removing tools while the retained conversation still describes their availability. Compaction-time instruction removal belongs to later work. The result text must say what actually happened, rather than presenting a pending marker as immediate revocation. Later MCP support will separately close live remote connections on unload; retaining instructions and retaining executable resources are not the same lifetime.

Deferred schemas do not defer authority

Portable rendering sends full declarations for enabled tools. An explicit Anthropic-only option supports tool search for a model the host knows can handle it. The host makes that capability assertion; a model-name heuristic cannot prove provider support.

Initial tools stay eager. Tools reachable through the primary's catalog but granted only later can have deferred declarations. A successful load records the newly granted names as result metadata, which Anthropic renders as tool references in the correctly paired loader result. Searching prose for names would lose that structure during replay or redaction.

The deferred schemas still travel in every request. This saves model context, not outbound bytes. More importantly, appearing in the catalog never grants dispatch. Deferral changes how the model discovers a schema; History still establishes whether the Agent may use it. Without a live catalog during offline rendering, the safe representation is the ordinary enabled declarations, not an invented reconstruction of missing manuals.

Source: Chapter 9.

Chapter 10 — Persistence and Replay

A restart should preserve the conversation without resurrecting the old runtime. Those are different operations. Human messages, completed answers, exact tool arguments, signatures, usage, redactions, and display identities belong to memory. Jobs, sockets, pause state, credentials, and live ownership relationships belong to the process that ended.

Persistence joins a reduced conversation snapshot with an optional diagnostic event journal. On loading, the snapshot establishes the remembered state and only a newer journal tail extends it. Today's configuration and manuals then establish what the new Agent can do.

Snapshot and journal answer different questions

The save document carries documentary configuration, reduced Context, an optional event array, and an `as_of` watermark. The watermark names the latest History sequence represented by Context. It is not a wall-clock timestamp or a claim that the diagnostic array contains every earlier event.

This supports several useful forms of the same memory. A full snapshot with events can explain both the effective conversation and how it developed. A snapshot alone preserves the

conversation without a large diagnostic log. An event-only document can reconstruct through the reducer. An older snapshot plus a newer event tail can advance the remembered state without reducing the older prefix twice.

Loading validates the whole candidate before replacing anything. Context must not contain entries beyond its watermark; event sequences must be positive and strictly increasing; supported part shapes, usage values, and relationships must decode. Explicit null is not silently treated as an omitted member.

Events at or below the snapshot watermark remain diagnostics. Only later ones enter reduction. The next event number follows the greatest valid sequence or watermark, even if retention has discarded the entire older journal. Sequence continuity belongs to the saved state, not to the current length of an array.

Runtime authority is supplied anew

Saved configuration explains the old session. It does not select the new vendor, endpoint, model, credentials, tool handlers, or primary manual. The host chooses these at startup. Saving credentials would be both unsafe and the wrong ownership model; core persistence never reads environment variables to recover them.

Runtime skill entries in the old document are structurally validated but do not restore authority. The loader removes old runtime instruction entries, restores dialogue and other effective conversation state, then appends the freshly rendered current primary closure at continuous sequence numbers. Ordinary

exchanges about previous skill loads remain part of the conversation, while old loadable grants do not silently become executable again.

This differs from offline replay with no runtime Skills manager. There, recorded rendered skill parts remain meaningful evidence of what the original Agent was taught. Runtime resume deliberately applies current policy. The same saved manual can be documentary evidence without being permission for a newly launched process.

Redactions and summaries survive as reduced state. A saved tool result that was replaced by a redaction does not grow its original bytes back during replay. Opaque replay content and exact numeric arguments likewise retain their representations rather than passing through a friendlier but lossy serializer.

Load nothing until a whole candidate is usable

The primary save and its backup use the same decoder as real loading. A file that parses as JSON but violates the save format is corrupt for recovery purposes.

Startup tries the primary first, then the backup if the primary is missing or invalid. If neither file exists, it begins a new session. If either exists but neither is usable, startup fails with the existing files untouched. This is deliberately stricter than Chapter 8's preference fallback: discarding unreadable font settings keeps an interface usable; silently discarding a conversation could convince the human that a different session had resumed.

Validation and reduction finish before live History is replaced. A bad late event cannot leave half a restored conversation behind. Successful recovery from the backup also does not rewrite the corrupt primary merely as a side effect of reading it.

Saving is careful about the backup for the same reason. The complete new save is written to a unique same-directory temporary file. An existing primary is decoded before it is copied into the backup path. If it is invalid, it must not overwrite a known-good backup. Only after preparation succeeds does replacement expose the new primary. Temporary files are cleaned up on failure, and failure remains visible.

Same-directory replacement prevents readers from seeing a partially written primary. It does not make several files one atomic transaction or promise recovery from every power-loss scenario.

Retaining less does not mean remembering less

Diagnostic retention controls the event array, not Context. Keeping all events, the newest few, or none leaves conversation, usage, redaction state, and watermark intact. Captured in-memory diagnostics are trimmed only after the save succeeds. A failed replacement must not destroy the very evidence needed to investigate it.

A newly opened diagnostic log begins with the retained or bootstrap prefix. It cannot fabricate events already discarded. Once retention has removed earlier events, that truncated log alone is insufficient to recover the full conversation; the save document is authoritative.

An optional archive is a dialogue-only JSONL projection made from Context before trimming. It excludes runtime skill instructions and reflects current summaries and redactions. It is neither a second recovery authority nor a promise to preserve

removed raw material forever. A later primary-save failure can leave this derivative archive newer than the save, because archive, primary, and backup do not share one multi-file transaction.

The distinction is practical: the snapshot says what the Agent remembers; diagnostics explain retained activity; the archive offers a convenient conversation projection. Choosing a smaller journal need not amputate memory, and a readable archive should not become an improvised restore format.

Shutdown is part of saving

Library persistence is opt-in, avoiding accidental sharing of a working-directory save among embedded Agents. The CLI and GUI opt in by default and use `save.json` unless configured otherwise. Independent sessions need independent directories or explicit paths.

Configuration occurs once, before actor admission. Replacing a live Agent's conversation concurrently is not authorized merely because a public load operation exists. Shutdown joins owned work and destroys Jobs while History remains attached, records completed shutdown, saves, and only then proceeds through logical destruction. The real ownership paths remain available for that final save. A save error cannot be renamed successful persistence.

The CLI drains accepted work at EOF; the GUI saves after its owned host lifetime ends. Neither can execute cleanup after a force kill. Recovery here covers what was saved, not work that existed only in a terminated process.

The GUI reconstructs finalized cards from Context and uses retained event facts for lifecycle and direct-tool diagnostics. A snapshot-only save therefore still produces a transcript. It preserves response and part identities, does not append synthetic

history, and does not restart old jobs. A saved shutdown describes the previous runtime, not an instruction to disable the new one. Memory survives precisely because its representation does not pretend the old process survived with it.

Source: Chapter 10.

Chapter 11 — MCP Tools and a Browser That Can Answer

Inside the core library, Common declarations and functional partial implementations compile into one assembly. They do not need a network-shaped abstraction to call one another. MCP introduces something genuinely different: another process or browser, with its own failures, request IDs, timing, and lifetime. Here a transport interface earns its keep.

The network gets a vote about availability and completion. It does not get to decide the Agent's permissions. A remote catalog describes what a server offers; the active skill determines which offerings become usable.

External peers, internal owners

A skill owns its MCP server records. Those records own clients, pending requests, and a transport that is either an outgoing stdio process or a browser WebSocket tunnel. Agent owns Tools, Tool owns Jobs, and History still records facts through the actor. A remote tool is an ordinary tool with an external implementation, not a parallel execution subsystem.

Core has no ASP.NET or browser dependency. The optional GUI supplies the browser transport. The real Common interface describes sending, receiving, cancellation, and destruction across that external boundary; it does not restore interfaces between every internal module.

JSON-RPC rides these transports with deliberate limits. Stdio stdout contains protocol messages, while stderr is drained separately. Request IDs correlate pending replies; unsupported IDs, malformed traffic, or exhausted bounded queues are faults rather than excuses to lose reliable replies. Destruction closes the owned transport, faults pending work, and joins readers and processes. An unknown response after cancellation may be safely late; an incoherent protocol stream is not.

Outgoing and incoming connections have opposite initialization responsibilities. An outgoing client initializes its peer and discovers tools before activation. An incoming service waits for its peer to initialize it and advertise readiness. Confusing the two creates two sides politely waiting forever. The stdio program inherits a configured working directory and a small credential-free environment, not the application's provider secrets; literal argument elements are not interpreted as a shell command.

Reentrancy is a progress requirement

Suppose a remote call cannot finish until it asks Ensemble to read a file. If that reverse request waits behind the actor's current turn, the turn waits for the call while the call waits for the turn. Nothing about asynchronous syntax alone breaks this cycle.

Reverse requests therefore enter a nonblocking actor control lane. The actor still checks the tool name, current grants, and arguments, then runs work through ordinary Jobs. A trusted transport is not a new grant. Enabled local tools can be discovered; remote tools are not reflected back as local reverse capabilities, and automatic snapshot tools remain excluded.

A successful reverse call produces a host observation. It does not fabricate an assistant tool-use exchange, because no model made that call. Expected denial and bad arguments become controlled failures, while invariant and persistence faults must remain faults. This keeps both progress and the historical explanation of progress honest.

Activation crosses the boundary only once

Loading a server-backed skill cannot perform all its work synchronously on the actor. Connections, handshakes, and discovery may wait on external systems. They also cannot publish instructions or grants from arbitrary I/O continuations.

The actor first validates the dependency closure and captures the variables and working directory needed for preparation. Preparation starts or claims transports and discovers catalogs away from the actor. A reliable commit message then returns to the actor, which revalidates cancellation, ownership, and name collisions, renders the frozen manuals, installs only explicitly allowed ordinary tools, and records one dependency-first activation.

Failure destroys and joins prepared connections without publishing a partial skill. Once commit is queued, preparation waits for its definite outcome; cancelling an observer must not destroy clients that the actor just successfully committed. The server's catalog never expands the manual's allowlist. Ambiguous or colliding tool names fail rather than selecting whichever connection answered first.

A server-backed primary requires explicit asynchronous initialization after persistence loading and diagnostic setup. Concurrent callers join one shared initialization; cancelling one

observer does not cancel startup for everyone. Until commit, the manager is unready and grants no provisional tools. User input and direct tool work are refused, while actor controls remain responsive. Startup is tracked control work, not a pretend model call with an invented ID.

Browser-backed startup has a necessary ordering wrinkle: the GUI must listen before the browser can connect. The primary dependency closure determines whether a browser wait is required. A stdio-only primary should not hang waiting for a tab it never needed.

Unload closes the selected skill's remote clients, processes, and Tools immediately. Its manual can still remain pending conversational removal under Chapter 9's policy. A retained explanation does not keep an executable connection alive, and independently owned dependency connections do not belong to the skill merely because it used them. Restart rebuilds these resources from current configuration; saved tool JSON cannot resurrect a process. Failed primary startup must not replace a previous good session save.

A result is richer than stdout

Ordinary MCP calls remain Jobs. A foreground observation timeout means the work is still running; it does not cancel the remote request. Explicit Job cancellation sends cancellation across the boundary. Waiting for a result does not transfer ownership of its lifetime to the observer.

MCP returns ordered content blocks. Supported text, image, and audio become the existing neutral parts, preserving order and the error flag. Inline media requires valid base64 and MIME

information. Unsupported resource blocks fail explicitly instead of disappearing from the answer or triggering an unrequested fetch.

The Job retains an atomic structured result separately from incremental process-output bytes. Diagnostic output is written before completion is published. The first completed ordinary observation consumes those structured parts; later observations do not replay them. Automatic collection has its own completion read so it does not steal the public observer's cursor.

Decoding supported media does not establish that every provider accepts it in a tool-result position. Provider rendering remains another boundary with its own explicit refusals. A successful codec cannot grant an adapter a capability it does not have.

Fresh context has a short life

Some tools supply changing context rather than model-chosen actions. Supported naming prefixes and discovery annotations identify per-turn or per-round collection. Per-turn values are reused only within that turn; per-round values are refreshed before each model request, including requests following tool results. Conflicting frequency declarations are rejected, and only the actual active owner's grants make a definition eligible.

These are not callable tools in disguise. They stay out of provider declarations, `$TOOLS`, deferred catalogs, and discovery references. Guessed calls and reverse requests cannot invoke them. Otherwise an automatic context mechanism would quietly broaden the public action surface.

Collection creates real Jobs and waits for completed values. A timeout-limited running notice is not the snapshot the next request needs. Controls remain responsive while collection waits.

Interrupt abandons that turn's wait without killing an independent Job; its eventual result remains inspectable but cannot leak into a later turn. Shutdown has separate authority to terminate and join owned work. Pause still blocks HTTP after collection completes.

The completed values join the current manual ephemera slot just before request freezing. This slot is replaced and consumed, not appended to durable dialogue. Rendering or HTTP failure must not strand stale context for the next unrelated request. A failed snapshot contributes a short diagnostic naming the tool and real Job handle, not failed output promoted to observed facts.

One browser, one claimed session

The browser's existing GUI WebSocket also carries enveloped MCP traffic. Agent identity comes from server replay and must match before either controls or RPC dispatch. One socket writer preserves ordering. MCP shutdown can close its own tunnel without closing unrelated conversation display traffic.

Connecting registers a transport; it does not install tools. Loading the browser skill claims the eligible registration, completes discovery, and commits its explicit allowlist. That registration is one-shot. An observer tab can watch the conversation but cannot answer another tab's calls. A delayed callback from an old connection stays tied to that connection instead of borrowing the newly connected socket.

The browser can report bounded visible DOM and usable selectors, click or fill a uniquely selected visible element, and expose or control its existing speech queue. Inspection precedes action, and a fresh inspection establishes its effect. A DOM snapshot is not pixel vision. Queueing speech is not evidence that

anyone heard it. Omitting password values is useful, but the rest of the page may still be sensitive. A real external boundary improves reach precisely because its evidence and authority remain explicit.

Source: Chapter 11.

Chapter 12: The Agent Sees Its Working Interface

An Agent can know its own environment only to the extent that the environment makes facts available. This chapter gives that idea practical machinery: recorded manuals, truthful startup gates, and a browser bridge that can inspect the page through the same MCP transport already carrying controls. There is no additional faculty of introspection. There are instructions, capabilities, and observations with identifiable sources.

That distinction is especially useful when the Agent tests its own interface. Source code says what the page was intended to do. A browser observation says something about what the running page currently does. Neither can substitute for the other.

A manual is part of the application

Every built-in tool has a contract beyond its schema. The Agent needs to understand the difference between accepted work and completed work, observation timeouts and cancellation, output limits and full results, and the handles needed to continue. A skill manual teaches those habits while granting the matching tools.

The shipped `ensemble` primary establishes ordinary operation. Other manuals add the browser bridge and GUI debugging behavior. The host must actually ship and resolve the directory containing them; a README mentioning a skill is not availability. CLI and GUI receive an explicit skill directory and

primary selection, with compiled-in defaults resolving the shipped tree. Core receives values rather than discovering the environment for itself.

Manual placement is equally substantive. The primary body renders once at creation, with loadable manuals later entering provider instruction context exactly once. A friendly load-result message alone would leave the manual in ordinary tool prose, where it has the wrong role and can disappear with result retention. The recorded rendered text explains what was taught even after a source file changes.

Templates need to advertise usable capabilities. A parser that cannot resolve the forms in the shipped manuals can accept a skill file while quietly breaking its teaching function. The current contract supports both bare and braced references, including the shipped `${TOOLS}` form, and leaves unknown names literal. This is about making the actual manual work, not adding a general templating language.

The embedding host can also supply an ordered initial skill set. Primary and initial skills are validated as one transaction before load events or grants are published. The CLI defaults to the primary and programming skill; an explicit CLI list replaces its default additions. The library does not silently inherit those CLI defaults. One invalid entry must not leave the earlier entries active.

Rendering uses the prospective capability set, including the tools being registered by the same skill. Otherwise a successfully loaded manual can describe yesterday's tool surface. The model-facing loader accepts a skill name, not new directories, grants, or remote registrations disguised as extra arguments.

Readiness is observable behavior

A wholly local initial skill closure retains synchronous initialization. Any remote dependency in the primary plus ordered initial skills requires asynchronous startup after persistence restoration and diagnostic setup. Until its transaction commits, the manager is unready. User input and direct tool calls are refused while controls remain responsive; the application must not advertise an Agent that has only half its required environment.

Any browser requirement in that complete initial closure requires the GUI host to start listening before the connection can be claimed. The complete closure determines that wait, not the primary alone. A set whose remote dependencies are all stdio-based needs no browser, and a CLI that cannot provide a required browser transport fails instead of waiting indefinitely. Startup cancellation or failure closes owned work without overwriting an earlier good save.

GUI text arriving during startup remains in a bounded host queue and enters the Agent in order only after readiness. New arrivals cannot overtake the backlog during drain. MCP replies and control traffic continue to flow while that queue waits. Otherwise the very browser messages needed to finish initialization could get trapped behind the readiness gate.

Later disconnect is a capability loss, not permission to operate on stale state. A running ordinary-primary Agent can lose the browser's Tools while retaining a manual pending removal. A newly connected browser supplies a new registration; the old session is not revived by changing a socket reference. Failure during initial-set activation is stricter: the required startup has not succeeded.

Looking through the bridge

The browser implements an MCP peer over the existing `/ws` connection. A single message path separates GUI envelopes from MCP envelopes, and replies remain associated with the connection that received each request. The request's Agent identity must match the server-provided identity. A second tab can observe without impersonating the active tool-serving tab.

The shipped browser offers visible-DOM inspection, click, text input, speech-queue inspection, and speech enqueueing. The `gui-debug` manual grants all five through its owned browser connection. Each tool still has one owner; depending on a capability does not justify constructing another copy of its handler.

Snapshots are bounded descriptions of visible DOM content with usable selectors and disclosed truncation. Click requires one visible match and cannot quietly navigate to a different origin. Input requires one visible editable text field; quotes, Unicode, spaces, and a string such as `{Enter}` stay literal text rather than becoming keyboard macros. Hidden, read-only, ambiguous, or unsuitable controls are refused.

A requested action is not proof of its result. A follow-up snapshot establishes the resulting field value or visible page state. Speech-queue state establishes what the page has queued, not audible quality or human hearing. The bridge does not attach a snapshot to every provider request merely because the browser is available. These observations are requested on demand. Inspection omits passwords, but choosing to expose the surrounding page still requires care.

The `gui-debug` skill state owns its browser connection. Its complete surface includes DOM snapshots, clicks, text input, inspection of the spoken-audio queue, and requesting speech.

Unloading must release the connection as well as remove those capabilities. Browser availability alone is not a reason to activate them.

The input field is not the cause

The retained first-use account describes a particularly instructive freeze. The GUI paused its coding Agent while a human typed. The browser driver filled the prompt programmatically, producing an input event that looked like typing. Submission then cleared the field by assigning its value, which produces no input event. The pause transition happened; the corresponding unpause path never did.

The first repair distinguished programmatic input and explicitly cleared pause on submission. A later native-browser check found another path to the same mistake. With speech enabled, an utterance ended while programmatically inserted text remained in the prompt. The speech callback looked at the nonempty field, reconstructed "human typing," and held the Agent paused again. The input listener had remembered origin for one event, but the later reader had forgotten it.

The current bridge marks MCP-origin input. The prompt listener retains a separate human-typing fact, and speech transitions combine that fact with the current speech cause instead of inferring intent from text length. Successful submission clears both the field and the human cause. Genuine human whitespace still counts as typing; programmatic text does not acquire human provenance merely because time or a speech callback has passed.

This also retains the earlier Agent-wide control rule. Another tab can explicitly unpause; an unchanged local cause must not immediately reassert a veto. A later real transition can send

another control. The cause is retained for correct scheduling, not converted into permanent ownership of the Agent.

Origin tracking does not authorize overwriting a human's unfinished draft. The manual requires preserving that input unless replacement is permitted. Correct event plumbing solves a scheduling defect; permission remains a separate question.

What the historical encounter established

The retained historical experiment used a separate GUI-only driver without filesystem or command tools. That restriction made the screen its evidence. Given source access, it could have answered interface questions by reading the intended implementation and missed the running failure entirely.

The driver also exposed an interactive-debugger deadlock. The command handler waited in a synchronous PTY read for process output to close. A debugger waiting at its prompt would never close it, and the observation routine that could recognize the prompt ran only after the handler returned. The escape hatch sat after the operation it needed to interrupt. The repair gave the running job and its reader a lifetime independent of the initial observation, matching the job-versus-wait distinction already central to this architecture.

These were earlier encounters, not fresh live-provider runs of the C# endpoint. Bill Cox supervised them, and the record distinguishes operational guidance from findings: a stale server holding the port needed human direction; truncation was noticed by the human; an unsafe rendering path emerged in subsequent source inspection. Crediting all of these to unaided model observation would erase the very information needed to assess the observer.

The speech-disabled input check also proved less than it first appeared to prove. Native speech start and end events exposed the second pause path. Those observations establish measured scheduling transitions, not subjective listening quality, and an earlier unexplained two-tab speech stall remains unexplained. Self-knowledge improves when these limits stay attached to the result. The valuable loop is concrete: inspect the running system, act through a declared capability, observe the consequence, and keep the cause and evidence intact.

Source: Chapter 12.

Chapter 13 — A listener who can only hear

The first-edition listener experiment was supposed to fail. A restricted agent asked the target to run a command against a nonexistent path and report the exit code. Successful tool-result bodies did not enter automatic speech, so the prediction was that the listener would hear the dispatch announcement and miss the answer.

It got the exit code right.

The retained transcript explained why: the dispatch announcement was followed by the model's own explanation of the failure. The raw tool result had never been spoken. The listener accurately reported the words and incorrectly concluded that their delivery proved the channel had no gap. Correct information had arrived by a different route.

Bill's scope decision clarified the intended interface: “I listen to your thinking, and that is enough.” Thinking and response text provide commentary; successful tool bodies stay quiet unless a person deliberately reads an artifact. Failed tools and provider errors must speak. This avoids turning a file read into an unsolicited audiobook, while making the model's explanation responsible for conveying useful results.

Source labels make that policy observable. A transcript needs to distinguish thinking, response text, tool announcements, errors, and deliberately requested speech. Thinking and errors also need spoken prefixes, because a person listening cannot inspect a JSON label.

Structure before phrases

Two entries in that historical transcript were lone backticks. Streaming seemed the obvious culprit: perhaps a fence had split across deltas. Feeding the entire block as one chunk still produced backticks.

The actual error came earlier. The pipeline split on newlines before filtering Markdown. Opening fence, body, and closing fence reached the filter separately, so no fragment contained a complete block. The repair resolves fences against the whole raw buffer first, replacing a complete block with “code block.” An unfinished fence waits for more input or an explicit final flush.

The current C# GUI's browser formatter follows that ordering. It preserves link labels but removes destinations, strips presentation markers, and expands identifiers such as `HTTPServer` and `max_tool_rounds` into speakable words. It handles a supported prose subset rather than pretending to be a complete Markdown parser. Inline code can carry essential meaning; a fenced implementation usually benefits from being named rather than recited.

A lone newline becomes whitespace. A blank line remains a paragraph boundary. Otherwise source wrapping invents pauses in the middle of sentences. Deltas can end inside words, links, or punctuation sequences, so the formatter retains raw text and releases stable phrases with sufficient lookahead. Explicit flushes release unfinished tails before a tool announcement, error, or final completion.

Each displayed part retains an offset into its raw source, not into the shorter filtered copy. Deltas append text; the authoritative final replaces the displayed value and releases only the unconsumed suffix. A final without prior deltas therefore speaks its whole text, while a duplicate final speaks nothing. The

historical implementation displayed a whole final and flushed an empty buffer, producing a perfectly readable answer that nobody could hear.

Restored parts remain silent, including through a later final. Resuming narration midway through a restored fence could expose code whose opening delimiter the speech cursor never saw. New live parts can speak normally.

What the transcript actually records

The page owns one active native utterance and a FIFO of future phrases. It records an utterance only when that phrase reaches the front of the queue, immediately before native synthesis and silent transcript mode diverge. Cancelling a queue must not publish its discarded paragraphs as speech already delivered.

This admission log includes ordering, timing, origin, and part identity. Transcript mode uses the same queue and formatter but skips synthesis. It is useful precisely because it observes the application rather than reconstructing an expected answer through a second text-cleaning path.

Admission is still not sound. Native callbacks can establish playback transitions; neither callbacks nor text records establish pronunciation, comfortable pacing, working speakers, or intelligibility to a particular listener. Those require authorized audio and a person listening. Keyboard navigation and general screen-reader usability are separate concerns.

Speech also affects tool pacing. Queued phrases count as busy, not merely the current utterance. The browser combines that speech cause, when the preference enables it, with human typing. Human whitespace counts as a draft. Programmatic input does not become human typing when a later speech callback examines the field.

Pause commands go out only when the combined local cause changes. Reasserting an unchanged pause on every callback would let one tab repeatedly defeat another tab's explicit unpause. Escape cancels speech without deleting the draft; typing may therefore continue to hold the Agent paused. Stopping the voice is not necessarily permission to proceed.

A listener without a side door

The separate `Examples/Listener` application constructs an empty Agent with exactly five tools: `say`, `listen`, `sleep`, `set_speech_speed`, and `save_history`. It installs neither builtins nor a Skills manager. A prompt asking the listener not to inspect the page would be a request for cooperation; dispatch refusing guessed inspection tools is an actual capability boundary.

Privately, the Session uses a browser connection to submit through the real form, observe turn completion, read admitted utterances, and acknowledge rate changes. The model receives no selectors, screenshots, DOM snapshots, arbitrary evaluation, target response text, or raw tool bodies. Target and listener have separate provider configurations; the browser receives no provider credential.

`say` marks the target pending before submission and refuses another turn while it remains pending. Only the real completion frame clears that state. A fleeting idle display before acknowledgement cannot masquerade as completion. `listen` advances its own hearing cursor, returning promptly for new utterances; a timeout with work pending is not idle. Saving semantic history does not consume that cursor and records typed prompts and heard entries, not the listener's reasoning transcript.

These tools deliberately wait for completion. The listener has no `wait_for_job`, so returning an ordinary running handle could leave a pending listen consuming entries that the model never receives. The existing Tool option makes dispatch await the real Job and return its complete structured result. It does not introduce another scheduler or result store. Cancellation of an observer and ownership of the underlying operation remain separate.

Session declarations and persistent state live in Common. Functional partial sections implement the channel and browser lifetime directly in the same application assembly. No callback bag is needed to connect those parts.

The listener owns a fresh GUI, browser profile, and private connection, never the human's working page. Shutdown joins the listener Agent's Jobs before disposing the connection and stopping the owned processes. EOF, signals, deadline, and failed startup take that same cleanup path. An earlier Go/Node listener closed its browser but left a sleep timer alive, delaying exit almost until the sleep ended. The lesson is ownership, not a C# performance comparison.

The current source reports offline component, actual-page, and owned-process checks against loopback provider fakes. Those checks expose missing final narration, misordered filters, escaped capabilities, and stranded work. They cannot supply the last observation: whether listening to this application for hours is pleasant.

Chapter 14 — The world's best context engineering, before breakfast

Bill had the design in his head for a couple of months. One Tuesday morning he spent about twenty minutes describing it while CodeRhapsody checked it against the code. He took a shower; the first commit landed at 7:42, before breakfast. He also described starting several new chats to recover from deteriorating context.

The useful idea beneath the title is modest: the context window is a working set, not the archive. A file result may answer one question and then occupy every subsequent request. The user's goal and the reasons for an edit have a different lifetime.

The current design removes tool traffic without asking a model to summarize away the conversation. Agent chooses cuts at safe actor boundaries, History records and applies them, and Engine renders the resulting Context. Workers neither trim History nor decide when the next model request starts. In C#, those responsibilities use Common declarations and direct partial implementations, not a new reduction service.

Tool traffic ages in two stages

The default byte target is 400,000. It derives a whole-result budget of one eighth and a call-and-stub budget of one sixteenth. These are serialized tool bytes, not tokens and not a hard cap on the conversation. Dialogue and manuals can keep growing.

When a band exceeds twice its budget, the actor selects an old prefix to bring it back toward one budget. First, full results become stubs while their calls and identities remain. Later, calls and stubs disappear together. The second stage cannot jump past a still-whole batch. Larger, less frequent cuts avoid continually rewriting the front of the request, although that alone proves no provider cache benefit.

Complete call/result batches are the unit of selection. Incomplete, orphaned, or ambiguous exchanges stop safe prefix selection rather than inviting the reducer to invent a missing result. A result becomes eligible only after a successful response has presented it to the model. Restoration conservatively presents retained results again before making new automatic cuts.

Every cut records its exact sequence bounds. Replay applies that fact, not today's policy. Raising the target after restart cannot resurrect bytes already removed, and lowering it cannot retroactively change an earlier decision.

An optional, explicitly enabled policy treats large results more aggressively. After a successful request presents a result larger than one hundredth of the target, the consuming response can call `keep_tool_results` to veto immediate stubbing. That decision happens once, after all companion tool outcomes are durable. A later response or restart must not reconsider a kept batch. Keeping defeats this immediate policy, not eventual budget pressure. A similarly named custom tool does not acquire the control's authority.

Stubs make a deliberately limited promise: bytes were removed. An earlier implementation cited output paths whose numeric handles were reused after restart. A plausible path then returned plausible output from the wrong command. An addressless stub is less convenient and considerably less misleading. Rereading a file may recover useful current information, but not necessarily its original contents; rerunning a command may have effects. Historical identity belongs in preserved events.

Keep the explanation and the manual

Once tool bytes leave context, useful visible narration becomes valuable working memory. A short explanation of why a file is being inspected and what the inspection established preserves more than an unexplained read/edit sequence. It need not expose private reasoning or narrate every keystroke. Nor does its survival make it permanently true: changing sources still need to be checked.

Skill manuals already belong once in provider instructions. Their load results are expendable traffic; their instructions are not. Unloading immediately revokes grants that no other active skill supplies, including deferred discovery references and automatic snapshots. Dispatch refuses a guessed unavailable name, and owned server connections close. The pending manual nevertheless remains in SYSTEM through tool clearing and handoff. Removing permission and retiring instructions are different operations.

The earlier positional `compress_context` experiment gave the model much broader destructive choice. Bill's account described repeated loss of most of the conversation, including the

goal near its beginning. A later whole-task handoff started a fresh instance from a document, making another durable-state path responsible for continuity.

`micro_handoff` is narrower. It records an ordinary successful tool result, waits for the batch's other outcomes, and then records a distinct handoff fact. Interrupted undispatched calls receive their exact-ID refusals before the cut. Independent Jobs continue; forgetting an observation does not cancel its work.

Applying the handoff removes tool traffic, opaque reasoning, and private replay signatures, including signatures attached to visible text. It retains visible dialogue, system entries, and manuals. Older active handoff notes disappear, and one new marked note takes their place. Its identity is structural, not a magic phrase at the start of prose. Engine projects it as ordinary system text only when building a provider request.

A useful note identifies what succeeded, what failed, where the current truth lives, and what remains. Failed approaches often prevent more wasted work than a catalog of successes. Original notes and removed private parts can remain in the archive even though they leave the working view.

Manual clearing uses the same actor boundary. It removes tool traffic after completed batch outcomes without resuming a paused turn or waiting for every independent Job. The GUI cannot accomplish that by deleting cards locally. A recorded cut triggers silent replay of the reduced Context so old cards and late partials cannot reappear, and refresh does not speak the conversation again.

Record before publishing

A shutdown snapshot cannot preserve work completed before a process is killed. History therefore owns a recovery journal as well as snapshots. Writers serialize through its existing save gate: compute the next fact, write and flush it, then publish the committed Context and sequence. Slow disk work does not hold the short state lock that readers need to capture the last committed view.

If append fails, nothing new becomes live, and subsequent append/save attempts refuse. Reusing a possibly half-written sequence would manufacture ambiguity. Cleanup must still join work and detach the Agent from its parent while preserving the original recording failure.

Recovery validates candidates independently: primary plus a contiguous journal suffix, otherwise backup plus its suffix. A journal beginning at sequence one can recover a session with no snapshot. An incomplete final record may be ignored with a diagnostic; malformed complete records, gaps, and duplicates are errors. A failed candidate never becomes a partly installed conversation.

The easily missed retention boundary is the backup. If the backup ends at 100, the primary at 150, and the journal at 160, deleting everything through 150 destroys recovery from the backup when the primary becomes corrupt. Saving validates the new snapshot and old primary before rotation, preserves a good backup, replaces the primary, and only then prunes records at or before the validated backup boundary, retaining every newer record. Without a valid backup, the journal remains intact. This is process-crash recovery, not a claim of hardware power-loss durability.

Reduction itself distinguishes an inapplicable known fact from corrupt input. A recognized cut with unusable bounds can be diagnosed and skipped while later facts apply. Unknown kinds, unsupported versions, malformed identities, and broken sequences still refuse decoding. A blanket catch would hide invariant failures rather than make replay robust.

An archive cannot be reconstructed from what remains

The optional archive contains original events, including tool payloads and the cuts that removed them from context. Before saving it, History merges available archive, journal, and diagnostic events and requires a contiguous sequence with identical overlaps. Missing early facts cannot be manufactured from the reduced snapshot. Turning archiving on late may therefore fail honestly rather than produce a file misleadingly called complete.

The live byte target has one authority: Agent. The GUI reads that value and publishes a settings change only after validation and saving succeed. There is a consequential interface difference: library and CLI zero disable automatic reduction, while GUI nonpositive values select the default. Sharing a parser merely to conceal that difference would not simplify the semantics.

The source's current checks use local and loopback paths; the breakfast implementation and earlier model experiments are historical, not fresh C# performance results. The design's durable contribution is more concrete than a memory claim: the system can explain which bytes left, why instructions survived, and how the same working state returns after a crash.

Chapter 15: The Invisible Invoice

Bill reported $2,475 in model spend on the first edition through its recall chapter, attributing a few hundred dollars to a caching bug. The answers arrived correctly and on time. Repeated input simply stopped receiving its discount.

Chapter 14 preserves useful request prefixes where possible. That does not establish that a provider caches them. Three questions need separate answers: did the application send a reusable beginning, did the provider report reading it from cache, and is the applicable price known?

Engine owns the instruments: an optional cache lens observes actual sends, provider rendering places supported markers, and a process-local meter aggregates reported usage. History retains canonical response counts, not a mutable dollar balance. Restoring yesterday's conversation must not charge yesterday's tokens to today's display.

Look at the bytes that left

The lens captures consecutive requests at transport admission, not whenever somebody renders a request offline. It keeps exact wire bytes, a marker-stripped prompt view, and a separate options view. Provider order matters: Anthropic's prompt is compared as tools, system, messages; Chat Completions uses

tools then messages; Gemini uses system instructions, tools, contents. JSON property order alone is not the provider's context order.

The diagnostic envelope is new, but the selected properties retain their lexical bytes. Parsing and reserializing them could change escapes, whitespace, key ordering, and number spelling, producing a tidy comparison of bytes that were never sent. Options such as model and routing are separate so they do not obscure prompt-prefix comparisons, but the original wire remains necessary to diagnose those changes.

Provider markers must disappear from the comparison because the stable-end marker advances as conversation grows. They must not disappear indiscriminately: a tool argument or schema can legitimately contain a field named `cache_control`. The implementation recognizes legal marker locations and copies the other byte spans unchanged.

The first-edition stripper matched one compact marker spelling. The actual formatter inserted spaces. Hand-built compact fixtures agreed with the broken stripper; a fixture produced by the real formatter exposed it. An instrument that inspects generated artifacts needs examples of those artifacts, not just examples of its author's expectation.

Each prediction travels with its own request as an immutable value. A shared “latest prediction” lets an old response arriving late borrow a newer request's measurement. Locking that variable prevents a data race without fixing the attribution error. The lens locks capture rotation and comparison, never the HTTP wait.

Capture failure is diagnostic failure, not inference failure. An unwritable directory disables that capture and reports the problem without altering the outgoing request. Captures omit

authorization headers but can still contain private prompts, files, and tool arguments; a diagnostic directory is not a publication destination.

Place markers after rendering

Cache boundaries belong in disposable provider JSON, after tool-call/result adjacency and turn merging. A neutral Context entry is not necessarily a legal final boundary. A final user message may even contain both stable material and request-only ephemera; marking the whole message would bank a tail designed to disappear.

The current Anthropic path explicitly opts in and uses at most four distinct boundaries: the last non-deferred tool declaration, nonempty SYSTEM instructions, the surviving handoff, and the last legal stable conversation block before ephemera. Coincident positions collapse. Empty text, thinking, and unknown opaque blocks are not targets, and deferred declarations cannot receive the tool marker.

The tool boundary can preserve an earlier prefix when SYSTEM changes. The rolling history boundary covers the growing conversation instead of stopping at its first user message. Loaded manuals still belong in SYSTEM even when activation changes the prefix; relocating instructions to improve a cache statistic would break the instruction model. A handoff likewise changes what remains. Markers cannot keep removed traffic alive.

These are route-specific contracts. At this chapter boundary, OpenAI uses Chat Completions with implicit caching and an optional routing key, not the historical explicit-breakpoint

experiments described in earlier manuscripts. Gemini retains its implicit path without a new cache-object manager. The routing key is neither a breakpoint nor a credential.

Stability is also not eligibility. Cache minimums are model- and route-specific token counts, not request-file byte counts. Zero reads might mean a small prompt, expiry, routing, changed content, or another provider decision. A fake can award a cache read to any fixture; it cannot establish the real service's threshold. Fixed-reference and rapidly growing-history experiments can warm differently without contradicting each other.

Counts have an authority; prices have conditions

Usage normalizes into four disjoint buckets: ordinary input, cache writes, cache reads, and output. Anthropic reports disjoint input buckets. OpenAI and Gemini prompt totals include cached reads, which must be subtracted to obtain ordinary input. Gemini output also accounts for separately reported thinking without charging it twice. Streaming usage updates can be cumulative, so they are not separate purchases.

Missing required usage stays unknown. An omitted optional cache bucket can mean zero; a malformed supplied value cannot. Overflow likewise yields unknown rather than a reassuring wrapped number. Session hit rate divides cache-read tokens by all input tokens, including the cold first request. It is neither the latest response's hit rate nor the lens's byte-prefix fraction.

Engine records each successfully completed send once. Offline rendering, fixture parsing, replay, failed HTTP, and cancelled exchanges do not advance this meter. A provider may

nevertheless charge for a request whose successful usage never reaches the client. The display is a token-price estimate, not an account balance or invoice.

The actual response identity determines pricing: vendor, response model, and service tier. Counts are grouped under those identities so switching models cannot reprice earlier work as if the new model had done it. The source deliberately verifies only narrow exact price rows rather than guessing from broad model-name prefixes. Missing tier, unknown model, unsupported charges, or unknown counts leave the estimate unknown. A documented price does not establish current model availability.

Dollars are derived with decimal arithmetic from counts and applicable rates. If any contributing row lacks a price, the session total stays unknown; a partial subtotal must not masquerade as the whole bill. Other providers can still show useful token counts without fictional tariffs.

One meter, several views

The GUI takes one coherent Engine snapshot. Its header combines session cost, latest-response all-input tokens, latest-response cache reads, and session hit rate. The browser formats acknowledged values; it owns neither an accumulator nor a price table. Two tabs and a refresh therefore cannot create different charges for the same work.

A new Engine starts with zero session totals and unknown latest-response counts, even when it restores a conversation. A failed request neither advances nor erases earlier successful usage. Usage notifications remain silent to the speech channel.

The current source establishes these client behaviors with offline and loopback exchanges, not a real C# cache grant or provider invoice. The distinction is useful operationally: the lens

locates changed bytes, reported usage shows what the provider said happened, and conservative pricing estimates the consequence without pretending to settle the bill.

Chapter 16: Leaving Anthropic for OpenAI

A provider migration asks an uncomfortable question: does the agent own its conversation, or merely keep a copy of somebody else's request format? Ensemble already has neutral History, provider renderers, and a transport boundary. Switching services still requires real work. It should not require rebuilding the scheduler or teaching every tool about authentication.

The motivation was a conflict over context control. Ensemble removes stale views, tool output, and selected replay material. Anthropic's newer preserved-thinking policy binds retained reasoning to the prefix that produced it; editing that prefix can invalidate later thinking. Compaction is not categorically forbidden. The conflict is retaining those artifacts while independently changing their earlier context.

Opaque reasoning also carries an authority problem. Research cited in the source recovered encrypted reasoning by replaying it through another model, including secrets absent from visible chat. OpenAI subsequently reported stronger cross-account and model-family protections and a closed replay pathway. Unreadable bytes are not automatically safe bytes: replay needs an authorized context, not merely encryption. Ensemble preserves compatible artifacts with provenance rather than treating them as portable assistant prose.

Credentials have an owner

The current C# design does not introduce an interface just because API keys and OAuth tokens both become bearer headers. Ensemble owns one concrete Credentials store; each Agent freezes its funding and account selection in Configuration. Engine follows its parent chain to obtain credentials immediately before HTTP. The renderer sees public capabilities, not tokens, and can render saved conversations offline without refreshing an account.

Common declares the shared types and callable partial members. Auth owns their implementations and protected storage. CLI and GUI manage their own temporary callback listeners, but they do not acquire independent token caches. The explicitly configured credential directory is private, created restrictively, and never shared with another application's store. Conversation snapshots, manuals, browser messages, and cache captures are not credential storage.

A local application cannot hide a shared client secret in its binary. Authorization Code with PKCE instead binds completion to initiation: the app sends a hash of a random verifier during authorization and the verifier itself during exchange. Intercepting the one-use code is insufficient. Fresh state binds the callback to the attempt; a nonce binds the signed identity to it.

Some protocol details are unforgiving. The callback uses literal `127.0.0.1` and `/auth/callback`; only the port varies. First registration starts with a bootstrap client ID, then persists the issued ID before exchanging the code. The bootstrap cannot perform that exchange. A failed exchange can therefore leave a valid pending registration without inventing an authenticated account, and an uncertain one-use exchange is not automatically retried.

An unsolicited callback with wrong state does not consume the active attempt. A matched denial or failed exchange does. The listener may need to serve several requests before the real callback arrives. Callback pages expose no code, state, account, or authorization URL. CLI prints the private URL in its human login command; GUI opens it in a separate user-initiated window rather than placing it in the main DOM or conversation. A blocked popup starts no login.

Identity is not permission to spend

The ID token must pass signature and claim validation, not merely decode as JSON. The narrow C# validator verifies trusted issuer, issued audience, timestamps, nonce, and subject continuity, rejecting ambiguous key selection. An account is not identified by matching email strings across registrations.

Even a valid identity does not authorize plan inference. The token response must grant the plan-usage scope. An identity-only login can be saved while inference remains disabled. Discovery metadata once omitted this scope from its advertised list; following that list alone produced a plausible login without permission to use the plan.

Refresh rotation adds concurrency. The old refresh token becomes unusable when its replacement is issued. Two callers renewing independently can turn a routine expiry into a suspected replay. Credentials serializes local operations and coordinates processes sharing its file, then reloads under the lock before deciding whether renewal is still needed. The second caller can use the replacement already saved by the first.

Both replacement tokens are saved atomically before either is used. That prevents a mixed local pair. It cannot recover a new token lost after server-side rotation but before local persistence.

Terminal refresh rejection clears tokens and retains registration; transient failures retain credentials and use bounded retry.

After an inference 401, Engine asks about the exact rejected bearer. Credentials reuses a newer grant or renews under the same coordination; Engine retries the same request body once. A second 401 ends the exchange. Failed streams and quota refusals are not invitations to repeat work or find another payer.

There are only two funding modes: API and ChatGPT plan. No automatic fallback connects them. Successful login saves plan selection with the grant; logout leaves that selection intact. A signed-out next launch refuses instead of noticing an ambient API key and quietly spending it. Explicit API selection can override a launch without rewriting the saved preference.

Local logout clears tokens. Sign-out clears locally first, then attempts remote revocation and reports whether it was confirmed. Destroying the credential owner is neither: it joins operations and releases resources without logging the person out. Account controls are human operations, not model tools, and they do not mutate the running Agent's immutable funding selection.

A stream is not yet a completed response

Responses changes the wire format, not the canonical conversation. Engine projects messages, correlated function calls and outputs, and compatible opaque reasoning into input items. Plan tools use a namespace; the returned call ID still binds the output to the exact call. A funding change alters disposable projection, not History. Unsupported media refuses explicitly rather than disappearing.

Chat Completions, Responses, and funding remain separate choices. API keys can use Responses. Plan mode requires its supported streaming route. Exact capability rows choose reasoning defaults and explanation policy; neither a model-name pattern nor account catalog visibility establishes renderer support, price, or permission.

The historical migration encountered a revealing stream: a function call was complete in an item-done event, but terminal completion contained an empty output array. A parser that looked only at the terminal array lost the call. A fake that redundantly repeated every item never exposed the bug.

The parser now retains completed items by output index. A nonempty terminal output is authoritative; an empty one can fall back to completed item-done values. Added skeletons and partial argument deltas cannot fill the gap. Only terminal success makes a response executable. A stream that looks promising and then fails authorizes no tools. Missing usage can leave accounting unknown without invalidating an otherwise completed response.

Display progress is less privileged. Text and public reasoning summaries can reach the page while generation continues. Explanation-before-tools policy uses actual arrival order, not a terminal array reordered after the fact. A tool-first response receives correlated refusals through the ordinary actor, with no tool effect. A public summary is not the ordinary explanation that policy requires.

Summaries are for the listener, not the replay cipher

Bill has no macular vision, reads code at about 80 words per minute, and listens to model output at about five times speaking rate. A summary delivered after the answer misses the chance to

interrupt a wrong approach while it develops.

Public summaries therefore have distinct typed events and part identities. They remain separate from assistant text, survive adjacency to tool calls, and can feed speech incrementally. Encrypted reasoning stays in compatible canonical replay data; it is neither browser text nor something to pronounce. A final can contain a suffix absent from its deltas, so the existing speech cursor catches up exactly once. Replay stays silent.

Historical probes demonstrated substantial streaming summaries, but a later successful plan run returned none despite doing reasoning. Its launcher waited for a summary before arranging EOF. The task finished; the process remained open until killed. Optional presentation content cannot be a completion or shutdown signal. The current selected C# row supports `auto` and explicit `off`, not every historical summary setting.

Responses usage flows into the existing Engine meter, with funding kept distinct. Plan tokens never acquire an API-dollar estimate. The cache lens must include Responses `input`: its historical omission compared tools while ignoring the conversation and produced reassuring nonsense. Even after repair, a stable predicted prefix did not establish a cache grant.

The source records local signed-issuer, component, host, and loopback transport checks. It also records a later authorized live session on October 29, 2026: Bill completed CLI login in his own browser, account discovery succeeded, and three isolated plan requests completed with `OK`. Those bounded requests reported zero cache reads and writes. They establish that session's login and inference, not live refresh, revocation, GUI login, audible quality, or a dollar charge.

The migration changed credential acquisition, encoding, and stream assembly. The Agent, Jobs, History, and tool loop remained the owners they had been before it.

Chapter 17: Giving the Agent the Web, and a Status Line

Bill wanted search he could carry between models. Community tools looked attractive: open source, no keys, no lock-in. Three manual calls to one candidate looked promising. Fifteen distinct queries succeeded only eleven times, and the failures offered almost no explanation. Another project's advertised MCP support existed in its Docker deployment, not the installed Python package.

Those historical measurements explain the choice of Firecrawl, not a permanent service ranking. The architectural decision is more durable: web access belongs to the Agent's tool boundary, not to one model vendor's search format. Status supplies the complementary view. A page tells the Agent what the outside world returned; Engine's meter tells it what its own completed requests consumed.

A new transport, not a new tool system

HTTP MCP implements the existing external transport interface. The MCP client still initializes the connection, routes replies by ID, discovers tools, and executes them through Jobs. HTTP adds independently cancellable POSTs, not another scheduler.

A request with an ID needs a matching JSON-RPC reply. An empty accepted acknowledgement can finish a notification, not a request. JSON and SSE replies use the same routing. An SSE

request can finish when its matching reply arrives, without waiting for a server that leaves the connection open. EOF without that reply is failure, not an empty success.

Initialization publishes the server's session header before exposing its result, because the next notification must already carry that header. Disposal cancels and joins outstanding HTTP work. Redirects and automatic retries are deliberately absent: a redirect could forward the bearer elsewhere, while a retry might repeat a remote effect. This bounded transport is not a resumable event-stream framework.

The host supplies named authentication values separately from manual variables. The library does not rummage through process environment, and secrets do not become instructions or saved History. Missing authentication is reported before an anonymous attempt rather than disguised as an authenticated low-quota connection.

An early passing local run missed one diagnostic path. Non-success HTTP bodies removed the configured bearer, but an HTTP-200 JSON-RPC error could echo it. Source review found the gap; real-HTTP JSON and SSE cases reproduced it. Redacting the decoded diagnostic field fixed the leak even when JSON escaping changed its spelling. Ordinary tool content was not globally rewritten to conceal the issue.

Discovery does not grant authority

The loadable web skill grants exactly search and scrape. A credential can reveal a much larger inventory, including expensive crawls, account modification, or billing queries. Discovering those tools must not authorize them. Ungranted names never enter the model's enabled definitions, and guessed calls refuse as unknown.

Loading publishes instructions and authority only after connection preparation succeeds. Unloading revokes this skill's grants and destroys its connection, without revoking another active skill's independent grant. As chapter 14 established, the pending manual can survive after its authority has gone.

The MCP client belongs to both the Agent's MCP manager and the activating Skill. Construction and destruction maintain both relationships, so unloading does not leave a detached catalog alive. Common declares these relationships; functional partial sections implement them directly in one C# assembly. The transport interface is a real external message boundary, not a reason to turn every owner relationship into a service interface.

The shipped primary matters. A historical test supplied a primary that allowed web access while the installed one did not. Progressive disclosure correctly refused the real user's load request. Another run fetched a page successfully but lacked `wait_for_job`; the model saw a running notice and could not collect the result. The fix was to expose the existing observation tool, not increase a timeout or invent another fetcher.

That corrected run fetched roughly 1.8 megabytes and showed the model a selected 12,000-byte portion with truncation disclosed. The fetched page, saved result, and delivered excerpt were different artifacts. The answer depended on the excerpt actually containing the relevant evidence.

A page cannot promote itself to SYSTEM

Retrieved text stays in the correlated tool result, regardless of what it asks. A page might claim to be an agent configuration file, demand a diagnostic shell command, or request private context in the next search. None of those statements has authority to change settings, load a skill, or run a command.

The two-tool grant reduces the available damage, but it cannot prevent exfiltration through an allowed search query. Bill's preference for deliberate, supervised searches addresses a risk the capability list cannot eliminate. Search and scrape send the necessary query or URL, not the conversation merely because a page requests it.

Failures also have layers. HTTP can succeed while the MCP tool reports an error. A successful scrape can retrieve an HTTP-404 page with its tool error flag unset. Page metadata, tool status, and transport status refer to different operations. Treating any one as the verdict for all three turns “Page Not Found” into confident documentation.

A web result is retrieved evidence, not a new instruction source. Status is a current meter snapshot, not another conversation owner. Both use existing tool traffic and chapter 14's working-context policy rather than inventing a permanent parallel history of pages or status banners. Useful conclusions still need their source and scope.

Five useful numbers, one actual owner

The status tool returns the selected model, latest-response counts, session counts, estimated cost, and cache hit rate. It performs no model request of its own. From the executing Job it follows Jobs to Agent, then reads one coherent Engine meter snapshot. The handler formats existing data; it owns no counter, pricing table, or saved-log scan.

The first-edition status task exposed a broken ownership path. The clean composition root existed but had no callers; the binary built a flat substitute and stapled individual getters onto objects.

A check searching for two architectural words kept passing while the actual relationship disappeared. That is a historical Go failure, not a pattern to port into C# with callback bags.

The same investigation found mixed-model costs computed using the currently selected model's price for all earlier tokens. Keeping counts rather than dollars was sensible, but incomplete: counts needed their response identity too. The current meter retains actual model, tier, and funding. Selecting a cheaper model cannot retroactively discount yesterday's requests. Any unpriced contribution keeps the session total unknown, even after returning to a priced model.

“Last response” means the completed response that requested the status tool. The answer consuming that tool result has not happened yet. A later GUI meter can legitimately be larger. Unknown counts and prices remain unknown; the cache rate uses cache reads divided by all input, including writes. A token-price estimate is not a web-service bill or a provider invoice.

Human model selection enters through the actor mailbox while idle. Busy or queued work refuses, as do active direct Engine sends—the public Engine API makes guarding only actor turns insufficient. Validation builds a new immutable Configuration before publication. The switch preserves conversation, Engine, meter, lens, provider, surface, account, and funding; it resets model-specific resolved identity and defaults. Restart still uses launch settings.

CLI and GUI project acknowledged selection rather than assuming a proposed name took effect. Capability rows govern local compatibility; account discovery governs what the account can see. Neither establishes price or permission to spend.

The source's current results cover real hosts and local HTTP peers, including corrected diagnostic redaction and model switching. Historical live search and browser sessions supply separate observations, not a fresh C# hosted-service trial. The

practical gain is immediate: the model can obtain current evidence and inspect measured work without either operation inventing a new authority inside the conversation.

Chapter 18: The Sandbox

In October 2025, Bill built a secrets manager around an approval dialog. Credentials stayed in an encrypted store until the Agent asked and he approved. The first time the Agent needed his sudo password, it ran `cat .` He followed the reasoning and clicked Approve before the implication landed. Three hours later, he found the password committed to git.

That is his reported historical incident, not a fresh attack against the C# implementation. Its useful detail is that approval worked. The human supplied the permission while missing the consequence. More persuasive wording on the same button would not have established a file boundary.

Chapter 17 completed the dangerous combination: private data, untrusted content, and external action. A poisoned page can ask an Agent to read a credential and send it elsewhere using legitimate tools. This chapter constrains those tools without pretending that a prompt can make hostile instructions mechanically unenforceable.

The host is trusted; the conversation is not

The binary needs its provider credential and an HTTP connection to the model. Confining the entire application as though those were accidental privileges misses the intended boundary. The

threat here is a poisoned conversation driving tools, not compromised host code.

Each Agent strongly owns one Sandbox. Common declares the relationship, persistent root and handle state, and callable operations; functional partial sections implement the same object. Tools reach it through Agent, Jobs through their owners, and Engine through its parent. There is no copied root in each tool and no interface wrapper joining declaration to implementation.

An empty root retains full access. A nonempty root must exist and confines file tools, file-backed media, Jobs output, and History persistence. Safe mode is a separate choice: it removes shell and job-control tools. Web authority separately controls whether the web-search skill can load at all. None of these choices is an instruction asking the model to behave as though a capability were absent.

Launch choices are immutable. Hosts translate their explicit settings; the library does not inspect ambient environment. Neither host changes the process-wide working directory. A model switch preserves the chosen policy.

The two enforcement mechanisms are deliberately different. File tools run inside trusted C# and use Sandbox's confined operations. A shell never calls that code. Commands and their descendants need an operating-system boundary too.

Resolving a name does not grant access forever

A path resolver follows existing links, appends missing components without creating them, and checks containment. An ordinary inward symlink can work; an outward link refuses. Relative-path containment avoids accepting a sibling such as

`project-old` merely because its name begins with `project`. Refusals report the requested spelling rather than disclosing the resolved outside target.

Resolution alone leaves a check/use race. Between validation and open, another process can replace a component with a symlink. The earlier answer to "where does this name point?" does not authorize whatever it points to now.

The current native file layer retains an open handle to the physical root. Actual operations repeat policy resolution and open each component relative to retained directory handles with no-follow semantics. The final open refuses a replacement link too. A race may make a legitimate operation fail; it must not trigger a fallback to ordinary host path access.

Editing retains one read/write handle across reading, exact-anchor validation, and replacement. New files use exclusive creation instead of checking existence and then racing to create. Directory enumeration returns names, not reusable access grants; every later descent or read goes through Sandbox again. Recursive search tracks canonical directories to avoid inward-link cycles.

Native details produced real C# failures. A runtime diagnostic FIFO inside the root made a blocking open hold the Sandbox gate and stall search. Acquisition now uses nonblocking opens and rejects non-seekable handles. Search skips refused or vanished entries while retaining ordinary matches. The Apple ARM64 calling convention also required special handling for creation permissions, and append needed the managed stream position initialized at the end. A declaration that compiled had not established either behavior.

The same owner covers less obvious paths. Engine resolves file-backed media and cache captures. Jobs confines both its output writer and later readback; protecting only creation would leave observation as another escape. History uses confined access

for snapshots, replacement files, backups, journals, archives, and logs while retaining its write-before-publication rules. An append refusal cannot publish a new event or reuse its sequence.

Relative host-state defaults live beneath the selected root. An explicit outside path refuses rather than being silently moved. Failed GUI settings persistence changes neither the old file nor the live value. Model-facing Job reports use rooted-relative output names; trusted host metadata can retain the real path without broadcasting it to the conversation.

The shell gets a kernel boundary

On macOS, Sandbox prepares a launch through the fixed `/usr/bin/sandbox-exec` path and an in-memory Seatbelt profile. It does not start or own the process. Job keeps responsibility for PTY, stdin, output, cancellation, process groups, and joining.

The profile denies filesystem reads, writes, and networking, then allows the root and the system material necessary for useful local commands. Ancestors receive metadata-only traversal, not sibling-directory reads. Toolchain support uses a narrow exception for immediate Darwin cache files, not writable access to the shared temporary directory. Physical paths matter: `/var` commonly resolves to `/private/var`, and a policy written for the wrong spelling can deny perfectly legitimate work.

The confined environment is rebuilt from a short allowlist. It supplies a fixed system path, ordinary locale and terminal settings, rooted home and temporary directories, and generated Git configuration. Startup hooks, injected libraries, arbitrary tokens, and inherited provider credentials do not ride along. Git

gets a minimal local identity without reading the user's configuration, whose includes, helpers, aliases, and signing commands can carry far more authority than a name and email.

Process control has its own injection risk. The trusted PTY bootstrap reports its process group through one bounded, strictly parsed startup line before command input and output are admitted. A writable `.pgid` file would let the command nominate an unrelated process for the host to signal. A malformed handshake refuses and stops the child.

The historical first confinement profile denied everything. Every escape failed—and so did reading a file inside the workspace. That apparent success was useless. The current source pairs refusals with useful local file, Python, Git staging, stdin, and cancellation behavior. A failed network request only establishes denial when the same owned target remains reachable outside the profile.

`sandbox-exec` is deprecated but is the mechanism exercised here. Confined commands require macOS; unsupported execution does not retry unconfined. The current native evidence is macOS ARM64, not a portable security claim.

Close the diagnostic side door

The host can correctly withhold its bearer from tools and still leak it through a readable API log. A provider may echo a credential in diagnostic data; redacting when `read_file` opens the log is too late because another reader can obtain the same bytes.

Known credentials are masked before diagnostic writers receive them. Ensemble's shared scrubber obtains configured keys through live Agent relationships, handles overlapping values longest-first and common JSON spellings, and covers labels as

well as payloads. A separate GUI log writer initially escaped both confinement and masking; observing the actual outside file exposed the missed path.

Plan bearers remain local to the Engine exchange rather than becoming Configuration fields or a new global token cache. The one permitted 401 retry can involve both rejected and rotated bearers; both pass through the trace scrubber for that exchange. Remote HTTP error bodies remain omitted at their source.

This does not rewrite successful provider text, tool content, or canonical History. Unknown secrets and arbitrary encodings remain outside known-value masking. Trusted skills, assets, explicitly selected credential storage, and extra MCP services are host inputs. A stdio MCP server does not automatically inherit the command profile. The shell's denied network is not a ban on every external action an Agent can request through its other granted tools.

Children can narrow, not widen

Policy derivation prepares for the fleet without constructing one. A child can inherit the root or choose a descendant, but cannot escape through an existing symlink. Validation can recognize a future directory without creating it; actual Agent construction still needs the root to exist. Safe mode combines restrictively, and web authority survives only if both parent and requested child permit it.

The important return value is the effective Configuration. A widening request produces a refusal *and still returns the narrowed value*. If a caller ignores the alarm, the wall remains. Returning the dangerous request beside an error would make every caller part of the enforcement mechanism.

The first edition demonstrated the difference between displayed policy and real authority. Its tool-removal function deleted a normalized name that was not the registry's actual key. `run_command` remained callable while safe mode could look enabled. Absence must reach dispatch, including a guessed call, rather than stop at the advertised tool list.

Current C# results exercise local hosts, rooted file effects, diagnostic masking, and the macOS command boundary. Earlier model-led sandbox sessions are historical evidence with their own scope. None excludes privileged mount changes, host compromise, or every possible escape. What this design provides is more precise: a defined set of file and command operations cannot acquire broader authority merely because the model supplies a convincing reason.

Chapter 19: The Ensemble

Bill could listen to one Agent's reasoning at five times speaking rate and redirect it between tool calls. Several Agents are different. Their work needs named owners, private durable state, and a result the parent can actually collect. A fleet is not several independent chat windows with a shared optimistic ending.

This chapter opens two boundaries: an existing application can host the GUI around an Agent it owns, and an Agent can delegate work to children it owns. Both expose the same question. Does the public boundary carry a complete lifetime, or did a test quietly supply the missing pieces?

Borrow the view, keep the Agent

The core assembly remains headless. CLI references it directly; the GUI adds ASP.NET and browser dependencies in a separate assembly. Common declarations still live with the core, and functional modules implement their matching partial members in that assembly. A public GUI package does not move implementation fragments of core partial types into another DLL.

A host passes an existing Agent to the public GUI entry point. It does not receive a hidden replacement with fresh History and unrelated tools. Browser prompts, approvals, settings, speech, and replies use the supplied Agent's actor and existing tool paths. The view can therefore mount under an application's route prefix without pretending to own the whole application.

The package embeds its real assets. A deployed program started in an unrelated directory does not need the repository's JavaScript tree nearby. An explicit disk override supports development without guessing from the current directory. An asset-only mount exposes the existing frontend modules; this chapter does not invent a new component API. Relative imports must work under the chosen mount instead of escaping to the application root.

The GUI borrows its Agent and, when supplied, its web host. Cancelling a view stops admission, resolves pending browser work, joins relays and subscriptions, and releases resources the view actually owns. It does not destroy the Agent or stop the caller's host. A standalone entry point owns more infrastructure, but borrowing an Agent still does not transfer its lifetime.

Replay reconstructs the view without starting inference or repeating speech. Serving a JavaScript file demonstrates deployment; rendering a usable interface is another observation. The current source includes outside consumers and isolated browser work, while leaving audible speech and human accessibility unclaimed by those checks.

A child has two parents

Ensemble strongly owns every live Agent. A delegated Agent also has the Agent that spawned it as a parent. The first relationship covers global resources and identity; the second covers work, questions, and recursive cleanup. A constructor establishes both through the parents' relationship operations. Destruction removes both directions and recursively closes descendants, Jobs, pending questions, subscriptions, and MCP clients without destroying any multiply owned object twice.

C# garbage collection cannot perform that logical destruction. Nor does a callback that can manufacture an Agent replace the actual parent relationship. The current implementation constructs children directly from their owning Agents, using Common's declarations and the functional partial bodies.

Each child receives a named directory below its parent's state, including its own conversation and `submit.md`. Names are path components, not paths. Descendants cannot escape through a separator, and two live children cannot claim the same address. Shutting a child down removes its live identity but preserves the work on disk.

Isolation matters even when children share a task workspace for deliberate collaboration. Their conversation and submission files must not overwrite each other. The historical ownership audit changed a child's data directory to its parent's and broke several fleet behaviors at once. That was one serious shared-state fault with several symptoms, not several independent defenses.

A child inherits the parent's construction choices and enabled primary capability. Its task text grants nothing. A reviewer launched with a custom submission shape does not silently impose that shape on its ordinary grandchild. Persistent choices have owners; ad hoc delegation arguments are not a new ambient policy.

“Done” is a durable event

A delegated Agent finishes by calling `submit`, not by producing a plausible final paragraph. The main human-facing Agent and named peers still finish in ordinary chat: they have no Agent parent waiting for a submitted result.

Default submission contains text. A parent can instead supply a bounded structural contract for a typed object. The same compiled contract generates the tool declaration and validates the actual arguments, including nested values. Unsupported schema features refuse at launch rather than claiming checks the implementation ignores. References do not fetch URLs or local files.

A custom result is the object itself, not JSON quoted inside a string or tucked under an extra default wrapper. Arrays, booleans, nulls, and nested fields survive validation, durable storage, joins, and readback. Invalid arguments leave the child unsubmitted so it can correct them. Structural validity says nothing about whether the described work is correct; the parent must inspect that work.

Persistence comes before publication. The child validates, writes the complete object to a restrictive same-directory temporary file, flushes it, and atomically replaces `submit.md`. Only then can a join observe submission. A failed write leaves the child unresolved. The file is one JSON object, not an append-only stream that needs a repair parser after a partial final row.

A child can receive replacement work. Accepted admission creates a new work generation and invalidates the old visible submission. An older turn cannot submit for the newer work, so yesterday's accepted answer cannot satisfy today's task. Old durable files may remain as evidence; they cannot resurrect current success after a restart or shutdown.

Parent submission is bottom-up. Any retained child without a current submission blocks it, including a failed child. The parent must resolve or shut down that child. Failure cannot become a fabricated empty success merely to unblock fan-in.

Even a published result is not quiescence. The submit tool may still need to append its result, and the child actor may still need to finish and save. A join can learn the answer before

disposal is safe. Cleanup waits for reliable completion and drain rather than interrupting the child inside its own submission callback.

A question must break the join

A child sometimes needs an answer before it can finish. `send_message_to_parent` records a question and blocks that tool until the parent replies. `submit` means completion; overloading it with questions once created a circular definition of done.

If a parent waiting in `join_agents` stayed asleep until submission, the child could wait forever for an answer from a parent waiting forever for the child. Fan-in therefore returns early with pending questions. The parent replies, then resumes the join. A sole pending question needs no copied opaque ID; several questions require an explicit selection rather than a guess.

Questions belong to the actual sender and recipient. Child notices enter as labeled tool-origin information, not system or developer instructions. The root's parent is the human for messaging purposes, so root questions go to the human inbox rather than waking the root to answer itself. Displaying the inbox does not consume a question.

A running wait is not the running work. When a human hint arrives, the actor records it and wakes the observer promptly, regardless of its callback timer. The underlying Job keeps the same handle and eventual result. A later wait can collect it after the Agent has dealt with the interruption.

Each Job owns a replaceable observation pulse. A message completes the pulse an active observer captured; later observers see a fresh one, not an old interruption lingering in a queue. This

is separate from execution cancellation and shutdown. An interrupt with no current observer does nothing.

The terminal must keep reading while the actor works. The first-edition command loop blocked input on the same execution path that was waiting for a tool. Its comment promised mid-turn steering; its structure made that impossible. Library tests could inject messages directly and never notice the human had no working wire to the mechanism.

Correct the protocol, then stop correcting

A successful delegated turn that ends without submission receives a framework correction. If unresolved children remain, it is told to join or shut them down; otherwise it is told to submit. The correction occurs at turn completion, not through a polling timer.

After exactly three consecutive framework wakes, the runtime stops prompting and escalates failure to the parent. Only durable submission resets the counter. New work can clear an old failure state without granting an endless fresh correction budget. An actual failed provider turn follows the error path immediately rather than spending three more requests asking a broken exchange to behave.

The historical exhausted child initially remained labeled running. That made a parent wait for work the framework had stopped trying to finish. Failed, with an error and no submission, is a much more useful state. Activity, failure, and submission are independent observations, not interchangeable colored badges.

Other first-edition failures came from the same seam: tests supplied the factory, human sink, and recursive child construction that the executable forgot. The first spawn failed,

human questions went nowhere, or the fleet stopped at one generation. Current direct ownership removes the factory machinery, but only exercising the actual host establishes that its users can reach the library's capabilities.

The source records current local C# runs through root, child, and grandchild, typed submissions, human replies, busy hints, and joined subtree cleanup. A historical live fleet adds a useful narrower observation: a hint reached the later provider request, but its requested marker never appeared in the final result. Delivery worked; compliance did not. The parent still has to read what came back.

Chapter 20: Managing the Swarm

A delegated child and a colleague are different relationships. A child has a parent, an assigned task, and a required result. A colleague may continue independently while the human moves between conversations. Making both look like tabs does not make their lifetimes interchangeable.

Chapter 19 supplied delegation. This chapter adds named top-level Agents sharing one Ensemble, with independent workspaces, conversations, provider settings, and runtime state. The browser can select which Agent receives a message without making that selection the ownership structure.

The implementation also corrects the title's most tempting implication. It does not introduce a global society in which every model discovers and commands every other model. Ordinary model-facing fleet operations remain scoped to the caller's direct children. The human and embedding application can create and select peers through a different boundary.

Names identify relationships

Agent names are immutable, with hierarchical identity derived from the actual parent path. A delegated child's name is local to its parent. This is not a mutable display label attached to a separately invented permanent UUID. The implementation keeps direct object relationships and resolves control targets through their owners.

The initial host Agent is host-configured and has the same sibling-home placement as other top-level Agents. It is not duplicated in the persistent peer registry. Other top-level peers have distinct names and workspaces; duplicate names and aliases of an already owned workspace are refused. A friendly label must not become permission for two Agents to write the same save and event files.

Creation is transactional. Configuration and persistence are prepared, current startup skills become ready, and only then does the root collection publish the new peer. Failure unwinds the provisional Agent and its registrations rather than leaving a selectable object that never finished construction.

Creation takes an explicit current source Agent. Missing model and skill overrides inherit from that source. The new peer receives its immutable runtime configuration snapshot and parsed skill templates, then constructs independent mutable Skills and MCP relationships. Ensemble keeps no defaults-Agent pointer. Sharing a template does not share a live grant, connection, or conversation.

Persist launch intent, not the process

The root's `ensemble.json` records immutable launch choices: each peer's name, model, primary skill, and creation time. Individual conversation saves live in their corresponding workspaces. These records have different jobs: one says how to reconstruct a peer, the other says what it remembers. Creating another peer must not overwrite an older peer's launch model with a later interactive model choice.

Named peers are reconstructed at launch. Current startup configuration determines their runtime authority; saved history does not resurrect credentials, old tool handlers, or unfinished

processes. Restoring the root collection likewise does not respawn a previous process's descendant tree. The new process starts with real new objects and explicit readiness.

Writes preserve this distinction. Saving a conversation should not require rediscovering the roster by scanning arbitrary directories. Updating launch records should not rewrite every peer's transcript. Workspace paths and identities are validated before registration, including collisions that would otherwise appear only after restoration.

Deletion removes the live peer and its launch record, after the application confirms the required work boundary. It retains the workspace, conversation save, and event files. Deleting a selectable peer is not permission to erase its history from disk.

The deletion operation refuses while unfinished descendant work still depends on the peer. It is not a shortcut around the fleet's bottom-up completion rules. Once deletion is permitted, teardown is joined and the registry is replaced. If replacement fails, the stopped registered peer remains available for a retry. A browser selection change cannot substitute for cleanup or repair a half-completed deletion.

Give the browser a tree, not another registry

The Agent tree represents Ensemble's real relationships. Its snapshot exposes canonical addresses derived from parent paths, names, and current status. An unnamed or detached Agent has no such address. An address is a routing description, not a permanent identity: deleting and recreating the same named path can produce a different object. Already admitted work retains the

actual object it concerns. Live updates let a viewer observe work elsewhere without switching conversations. The tree is a projection, not a second place that owns Agents.

Selecting another Agent changes the target for input, hints, pause, interrupt, settings, and transcript viewing. It does not pause the old selection or cancel its background jobs. Each control carries its actual target; a delayed click must not act on whichever Agent happens to be selected when the server receives it.

The browser bridge has a separate connection boundary. Its existing MCP relay stays with the Agent that established it; selecting another conversation does not transfer or renegotiate that connection. Peer selection and browser capability ownership must not be conflated.

Recorded transcript state and current runtime observation remain different. Reconnection rebuilds the selected view from authoritative state and resumes updates; it does not infer completion from the absence of recent chunks. A quiet peer may be waiting for a provider, and a peer with visible final text may still have obligations to children.

Identity is also essential for delayed observations. A result from an old selection still belongs to that Agent. Reusing one mutable “current Agent” pointer when displaying callbacks would make the interface fast and wrong.

Delegation keeps its existing boundary

The fleet tools still act on children owned by their caller. Spawning creates that parent-child relationship. Waiting, messaging, inspecting status, and shutting down operate on the corresponding child handles. Knowing a top-level peer's ID does not grant a model the right to control it.

Child questions remain request/response operations. The parent can answer the exact question while ordinary child output continues independently. A running observation is not submission, and a child that has produced a reply is not necessarily finished with its task. Submission remains a validated, durable event rather than a guess based on conversational tone.

The observation tools answer different questions. Sending a message steers active work or admits a turn when idle. Asking a child instead admits a separate turn and waits on that exact reliable reply; an old assistant final cannot satisfy it. Waiting for finalized text or an idle status is not waiting for typed submission. Reading a saved submission is not the same operation as joining current selected-child completion.

A change wait captures the current child objects and their observation baselines at admission. It notices status, committed output, questions, and transient streaming progress without taking messages away from another observer. A later sibling does not silently join the wait, and a replacement with the same local name does not become the old captured object. Lack of observed progress is a useful supervisory hint, not proof of deadlock. An observation timeout leaves the work alive.

The parent and child lifecycles also remain visible. Parent shutdown must cancel and join owned work before destroying resources it uses. A display card disappearing does not establish that the work stopped. Conversely, retaining a saved child directory does not keep the corresponding live Agent alive.

This boundary is intentionally narrower than arbitrary peer-to-peer messaging. A global broker would need its own authority, discovery, and lifetime rules. The current application has no reason to acquire those mechanisms merely because multiple Agents appear in one window.

Why immutable configuration matters here

The pressure to share mutable settings increases as soon as several Agents exist. A host already has a convenient configuration object; passing it to each new Agent seems cheaper than resolving a snapshot.

That shortcut makes later behavior difficult to explain. A model change in one pane might alter another Agent's next request. A parent's updated policy might silently affect a child midway through work. Restoration might mix the old peer's saved intent with the currently selected peer's provider route.

Resolved snapshots remove that ambiguity. Deliberate settings changes still go through the owning Agent and apply at their documented boundary. They do not travel accidentally through an alias to somebody else's settings.

The same discipline applies to observers. The browser can receive a merged view, but each underlying event still identifies its owner. Aggregation is useful precisely because it does not erase the facts that make the individual streams intelligible.

Parallel work is only useful if its cost is legible

A second Agent can investigate a different failure, review a draft, or work in a separate workspace while the first continues. It can also repeat the first Agent's work, wait forever for an answer already sent elsewhere, or spend tokens solving an unassigned problem.

The application cannot make delegation useful by multiplying model calls. It can make the assignment, ownership, status, and completion visible enough for a human or parent Agent to manage them. Explicit submission, questions, cost observations, and retained traces are the practical control surface.

This chapter therefore adds a modest thing with substantial consequences: several independent roots inside one usable application, and an honest view of the children they own. The next problem is temporal rather than spatial. Even a correctly identified Agent eventually runs out of room to remember why its work matters.

Implementation chapter

Chapter 21: Forgetting on Purpose

Bill once took an agent experiment that worked remarkably well at home into his workplace, where it failed. He initially blamed the model. The important difference turned out to be a bug: memory worked at home and was completely broken at work. Fixing memory reversed the apparent comparison between models.

That historical account motivates a feature; it is not a controlled benchmark. The engineering problem is concrete enough without one. A new session can have the source code and still lack the reasons for it. A summary can preserve those reasons, but accepting an unreliable summary must not destroy the only usable account.

Memory therefore makes forgetting a replacement transaction. The old material remains active until a bounded worker's proposal has been validated and committed. History, working Context, human-readable files, and the model's judgment have separate roles throughout.

Keep different rates of change apart

Context presents five bands from stable to recent: SOUL, MEMORY, the 64x archive, the 8x archive, and session memory. Conversation comes afterward. Compression flows in the opposite direction, from completed conversation into sessions and then through progressively coarser accounts.

The names describe desired resolution, not a promise that a model will compress every input by exactly eight. SOUL is outside automatic inbound graduation. Making identity available to the model does not authorize the compressor to rewrite it.

On disk, the bands use `SOUL.md`, `MEMORY.md`, `bucket-1/`, `bucket-0/`, and `memory/`. An Agent owns Memory, which reaches rooted files through that Agent's Sandbox. The worker does not choose a destination path. A library Agent without a home does not quietly acquire filesystem memory somewhere convenient.

Budgets here are UTF-8 bytes, not estimated tokens. The default conversation budget is 3,000 bytes, with 1,500 for each corpus band; high and low watermarks default to twice and once the resolved budget. These are trigger and writing policies, not hard promises about every result's length. In particular, the target byte budget does not turn an otherwise completed, nonblank candidate into a failed submission merely for exceeding that target.

Six immutable policies govern conversation and the five bands. Disabled bands leave disk intact while retiring their active projection. Re-enabling reads the current corpus. Policy saves precede publication, and acknowledgements describe what the selected Agent actually applied. A preference change cannot safely race a worker that captured a different source or destination policy.

A checkpoint preserves the boundary

A handoff still removes old tool traffic and private replay from working context. It also leaves a readable continuation note. Earlier handoff notes now remain as completed-segment boundaries until a matching memory replacement commits.

When retained conversation exceeds its budget, selection takes a whole oldest completed prefix, stopping when the remainder fits or no further complete prefix is available. It never grabs the unfinished tail or pretends an unresolved tool call has a result. Selection itself changes nothing.

File graduation has a different condition: it requires at least two source files. Strict dated identities, deterministic ordering, and overlap checks establish which files are active. A directory enumeration is not a chronology, and a convenient Markdown filename is not evidence that a staged candidate was accepted.

The archive remains more detailed than the request. History preserves the canonical events; Context is a reduced working view. Memory renders as labeled user/data material before conversation. It cannot grant a tool, become a system instruction, or turn a remembered claim into current authority.

Capture what the replacement means

An attempt records its task identity, exact selected source, prior destination bytes, fingerprint, and child trace path. Later unrelated dialogue may arrive without invalidating the capture. Editing a selected file or the destination does invalidate it.

Comparing a whole-History version would discard useful work whenever the user typed another sentence. Comparing only a task ID would accept a summary of bytes that no longer exist in that form. The capture identifies precisely what the candidate is allowed to replace.

The actor records the launch before starting its one owned compressor. Parent work can continue while the child thinks, but acceptance waits for a complete main-work boundary. The child returns immutable data through the mailbox; it never edits parent History directly.

Relevant policy changes cancel and join the attempt. Failure retains source and waits for a later checkpoint instead of immediately retrying forever. A successful commit may advance the next eligible file graduation. These transitions belong in the existing actor, not a second scheduler polling for signs of readiness.

The compressor has a small job and a real end

An early direct implementation made one Engine call and expected a finished handoff. The model needed another turn, returned no final text, and the attempt failed. Unlimited calls would have hidden the missing completion rule rather than supplied one.

The current worker is an ordinary child with both owning parents, but without inherited skills, recursive memory, file access, shell access, or web tools. Its only tool is `submit`, accepting one required string named `memory`. The prompt supplies the source and existing destination as data. The model proposes text; the owner controls files and acceptance.

The actor permits at most three Engine sends and three tool batches. A batch containing several calls is refused before any runs. After accepted submission, the turn ends without asking for closing prose. Ordinary final text is not submission. Blank output is a terminal failed candidate; malformed shapes can consume the remaining bounded correction opportunities. Truncated or incomplete provider output cannot smuggle a plausible-looking submission through the terminal boundary.

The prompt asks for decisions, reasons, failures, uncertainty, and unfinished work. It asks for first-person continuation only where the source supports it, with attribution for somebody else's experience. No runtime check can prove that a fluent summary preserved the right judgment. That still requires reading the result and testing what later work can recover.

Prepare, commit replay, finish disk work

Acceptance first rechecks source and destination, stages a hidden candidate and receipt, and archives canonical events for a conversation cut. After that I/O, it revalidates before publishing the replacement.

The actor then records task-correlated removal and matching population. Removal alone holds a proposed cut; it does not make the source disappear. Matching population applies the held removal and installs the actual replacement text in one Context transition. Skill entries inside a conversation interval survive; a generic destructive range cut would remove too much.

Only then does recovery promote the staged file and retire exactly the captured inputs. A successful terminal outcome is recorded as `Committed`, after which the receipt can be released. File finalization is an operation, not another success state. Retired files remain available on disk, excluded from active scans but available for later recall.

These steps cannot be one filesystem rename. A crash may leave a launch without a candidate, held removal without population, committed replay without file promotion, or a

promoted destination with retirement unfinished. The receipt and replay state distinguish those cases. Startup resolves them before scanning the corpus.

Recovery completes the same accepted decision without asking another model what the summary should have been. Missing or conflicting required evidence causes refusal, not a guessed repair. Repeating finalization must neither overwrite unrelated edits nor apply an accepted transaction twice.

That is a bounded process-crash contract, not a promise of filesystem-wide atomicity against a hostile writer or every kind of power loss. The important invariant is that accepted replay, the live projection, and the corpus have a documented route back into agreement.

Spending is not occupancy

The compressor's work costs money even when its result is unusable. After joining the child, the parent imports measured model, tier, billing, and usage rows exactly once. Unknown counts remain unknown. Replay of documentary usage does not charge the new process again.

Those rows must not replace the parent's latest direct-response observation or cache lens. Cumulative spending and the main conversation's current occupancy answer different questions.

Context pressure uses the latest actor-accepted main response and a known window for the selected model. Provider accounting differs: OpenAI cached input is included in its total input, while Anthropic exposes separate categories. A child's response or a session-wide token sum is not evidence about the main request's occupancy. Unknown limits remain unknown rather than being guessed from a model name.

At 90% occupancy, a request-only instruction asks for a handoff. At 95%, the runtime latches a checkpoint requirement: both declarations and dispatch admit only an already-authorized `micro_handoff`. It does not grant that tool merely because it would be useful. A later short or usage-less response does not silently clear the outstanding requirement.

If a compressor already owns the source needed to regain headroom, the actor waits for that exact attempt before another main request, while continuing to process controls. A matching committed replacement releases the wait. Failed or invalidated work produces an explicit headroom error, not another hopeful request into an already pressured context.

Keep one owner for each fact

The memory cache boundary follows the most recent surviving handoff, or the final memory block when no handoff survives. That structural choice supports stable prefixes; it does not guarantee a provider cache hit.

The broader discipline is the same. Files supply corpus bytes through Memory. Accepted events supply replayable state. Engine renders the provided Context. Rendering does not rescan files, launch compressors, or decide which history should vanish.

The original source remains available long enough to reject a bad proposal, recover interrupted finalization, or retrieve a detail the summary omitted. The next chapter gives the Agent a habit of looking for those details before it repeats a mistake.

Implementation chapter

Chapter 22: Auto-Recall

Bill's earlier memory system had a search tool and an archive full of useful decisions. The agent rarely searched. It could produce excellent answers when it did, then begin the next session by proposing an architecture the team had already rejected.

Availability had not produced a habit. Automatic recall moves the initial search to new-turn admission, where a relevant old fact can arrive before the main model answers. The model still has to use the fact well. It no longer has to remember to look for it first.

The historical improvement in daily use motivates the design. The C# pipeline adds explicit ownership, cancellation, replay, and input-boundary rules so that a helpful retrieval feature does not become an unaccountable source of instructions.

Search several small worlds

Five independent corpora cover memories, documents, handoffs, learnings, and visible skill manuals. Memory includes live and retired originals as well as SOUL and MEMORY. Skill discovery reads exact `SKILL.md` files, not every script and fixture under a skills directory.

Separate indexes keep a large documentation collection from deciding what counts as a rare word everywhere else. They do not turn scores into calibrated probabilities. They keep the statistics meaningful within each source.

Each Agent owns a Recaller whose indexes are derived immutable values. Searches read current bytes, fingerprint names and content, and replace indexes only after a complete successful refresh. Equal file sizes do not establish unchanged text. Readable invalid UTF-8 or oversized input refuses the refresh; quietly accepting a partial corpus would conceal a broken input boundary.

Agent-home traversal uses its Sandbox. An explicitly selected skill tree has its own provenance and a read-only rooted handle, without creating broader file-tool permission. Memory's recovery gate prevents recall from indexing a half-promoted transaction. A staged candidate must not become evidence merely because it resembles a Markdown file.

These rules permit stable memory files in the corpus without blindly attaching them twice. Duplication is determined against the actual retained and projected context, not by categorically excluding useful files from search.

BM25 earns its place

For a small Markdown archive, local keyword ranking avoids an embedding service, another database, and another source of credentials and latency.

BM25 combines term frequency with saturation, rarity within the corpus, and document-length normalization. Repeating a word ten times should not make it ten times more useful. A term found everywhere should contribute less than a distinctive project name. A long document should not win merely because it has more opportunities to match.

One apparently minor choice caused a particularly quiet failure. Classic inverse document frequency can become negative when a term occurs in more than half the documents. Clamp that

value to zero in a small archive, and every useful query term may contribute nothing. The index builds and search completes, but the Agent never remembers anything.

The chosen smoothed form, `ln(1 + (N − df + 0.5) / (df + 0.5))`, preserves a positive contribution for a matching term. Thresholds still matter; a positive score is not automatically a selected result. Tiny fixtures can also make useful terms commonplace, so the test corpus must represent the competition it claims to exercise.

A test that merely finds a planted word is weak. Raw term frequency can pass it without implementing BM25. Better cases pit a rare term against common repetition, several distinct query terms against one repeated word, and a focused note against a long document with one incidental match.

Markdown is chunked at headings, with smaller boundaries for oversized sections. Breadcrumb headings retain the location of a paragraph. The source text and provenance remain available; the index does not become another authoritative copy of the archive.

Personal memory needs a selection policy

A decision recorded in a short daily log may matter more than a long design document rich in matching terminology. Independent indexes alone cannot express that preference.

Mechanical selection reserves half the final slots, rounded up, for qualifying memory hits. Other sources divide the remainder, and unused slots flow to sources with more results. The

reservation applies to final selection. Capping a candidate pool and then globally sorting it can still let another source take every final slot.

The full ranked pool remains available for fallback. The smaller candidate limit exists to bound a judge request, not to throw away memories before mechanical quotas can operate.

A semantic judge may override this crude policy. When it successfully selects nothing, the result is nothing. Treating an empty verdict as failure would undo the judgment the application paid for.

The judge is a call, not another agent

The judge makes one stateless request through the existing Engine. It receives the current query, a small amount of ordinary recent dialogue, and numbered candidate snippets as framed data. The current query alone supplied the search terms; recent dialogue helps interpret the candidates.

There is no child Agent, second transcript, recursive recall, skill catalog, or tool authority. The default judge uses the current model and funding route; an optional same-provider model choice does not establish account entitlement or mutate the main Agent's configuration.

Recall owns both the response-format instruction and its parser. Engine owns transport and provider-native decoding. Splitting the answer contract between those layers invites a model to obey one format while the other rejects it.

Only completed, tool-free output can supply a verdict. Valid array syntax with no usable indices selects nothing; malformed syntax is failure. Duplicate or invalid indices do not silently become the first candidate. A plausible index found in a truncated stream is not completed output.

The judge has an operation-level timeout that reaches the HTTP request. Returning from a wait while a stalled stream runs forever would leave the expensive part alive. Timeout, transport failure, malformed output, or incomplete response falls back to mechanical selection. Cancellation of the user turn instead propagates; it is not permission to keep working and publish a later fallback.

Measured usage enters cumulative accounting once without changing the main latest-response, pressure, or cache observations. Replaying an attachment does not purchase another judgment.

Keep the human's words intact

An ordinary Human turn is recorded exactly as admitted. Recall then runs as foreground work under the existing actor. Busy hints keep their current immediate behavior rather than starting a second recall alongside it.

The worker searches and judges, then returns full snippets. The actor joins it, verifies the exact turn identity, and checks duplicates again against current context before publication. A hint received while the judge waits may already contain the proposed fact. An interrupted turn must not donate its recall to the next one.

Accepted recall is a separate `RecallAdded` fact with a `RecallPart`. It becomes Human/data content without rewriting the original input. Quoted text, filenames, headings, and attribution make the origin visible. A recalled document saying “ignore the user” is still a document; a recalled skill description does not load a skill.

The recorded attachment stays available for short follow-ups. Rendering does not search today's files and rewrite yesterday's conversation. The selected bytes and identities survive ordinary save and replay.

Bound display without losing identity

The complete formatted attachment has a UTF-8 byte cap, including headings and quote prefixes, and ends on a valid rune boundary. That bounds the request without pretending a truncated display is a different source.

Deduplication therefore retains hashes of the full selected snippets alongside any shortened presentation. Otherwise a long note can be attached again on the next turn: the retained prefix does not contain the omitted suffix, so full-text matching alone fails.

The check considers retained recall, readable dialogue, nested tool results, live memory, and the instructions actually projected for the provider. Opaque replay is not treated as readable text. A hash is recorded only when some source content fits in the attachment.

Those identities belong to retained context, not an eternal “seen” table. Once a committed cut removes the entry, the fact may legitimately be needed again. A handoff alone no longer releases it while Memory is still holding the source for an uncommitted replacement. Clearing the identity early would permit a duplicate before the safe cut occurred.

Retrieval remains a fallible assistant

Keyword matching misses paraphrases, chronology can matter more than word overlap, and a judge can choose badly. Historical tuning is useful experience, not a guarantee of current provider relevance.

The implementation can nevertheless make the failure legible. A failed judge can fall back; a corrupt corpus cannot be silently called “no matches.” Settings are saved before publication, and a turn uses its admitted policy rather than half of a later GUI edit. Skill descriptions can help discovery without granting their capabilities.

The practical gain is a more informed starting point. An old reason arrives before the model confidently reinvents the decision it once helped reject. The next chapter examines how to affordably carry that growing, structured past through provider requests.

Implementation chapter

Chapter 23: While the Cache Is Broken — Measure Before Taking the Detour

A long-lived Agent sends much of the same conversation repeatedly. Stable instructions, memory, and accepted dialogue can form a reusable prefix while new work grows at the end. Small changes near the front can instead make apparently similar requests expensive again.

Caching is therefore partly a conversation-design problem. It is also a provider measurement problem. A client can control the bytes it sends; it cannot manufacture a provider-reported cache hit or infer one from a quick response.

Separate intention from observation

A cache key may influence provider routing, but it does not promise warm entries, stable server placement, or a particular reuse policy. A catalog price can estimate the value of credited input; it cannot establish that an account is entitled to send a request.

Local prefix information and provider usage consequently remain separate observations. Comparing two rendered requests can reveal how much shared material survived. Native usage can

report what the provider credited. When reuse is absent or unknown, the display must say that rather than substituting the local estimate.

This distinction is especially useful when debugging. A locally stable prefix with no reported reuse suggests a different question from a request whose first instruction changes every round. Neither observation alone identifies the provider's internal cause.

Keep the front stable for a reason

System instructions precede stable memory, which precedes recent conversation. Loaded manuals are recorded once, and changing a source file does not silently rewrite yesterday's active instructions. Recall attaches at turn admission instead of being recomputed on every tool round. These choices improve replay as well as prefix stability.

Handoff provides a structural cache boundary. When a surviving handoff exists, its position defines the cut; otherwise the last memory block can supply it. Without either, the application invents no boundary. Provider adapters translate that decision only where their protocol supports it. A Responses request must not inherit a Chat-specific marker merely because the desired optimization is similar.

Correctness still comes first. A committed memory replacement changes context because the older source has been safely replaced. Avoiding that change to preserve a hypothetical hit would turn an optimization into an owner of conversation state.

Freeze one logical request

Provider activity has two related objects: the logical request the application intends to make, and the transport attempts needed to send it. A credential retry must not become an opportunity to quietly choose a different model, endpoint, funding source, tool set, or replay context.

Capture request configuration and credential-source identity before the attempt. Record actual wire behavior and the accepted response. Failed attempts can have diagnostics without acquiring authoritative completion or usage that the provider never supplied.

Secret-bearing diagnostics need their own handling. Making requests inspectable does not authorize copying credentials into a benchmark artifact. The useful evidence is the route, redacted request shape, provider outcome, and measured usage—not the reusable secret that permitted the call.

Make a cache experiment falsifiable

The local benchmark fixture freezes prompt and configuration, captures the actual outgoing request, and makes differing provider usage reports observable. It can prove request stability, adapter behavior, and accounting semantics. It cannot establish a commercial service's current caching behavior.

The bounded public Responses series in the source accepted repeated identical requests and reported no input reuse. That is a valid measurement: the requests reached the service, but the reported outcome did not support a reuse claim. It does not explain why the provider withheld reuse or prove that the route never caches.

A separate explicitly enabled subscription-route exercise reported both hits and a miss. That result establishes the bounded path and its observations, not reliable reuse on every future request or a guarantee about subscription cost. A useful report retains both outcomes rather than treating a single hit as a stable operating guarantee.

A denied model request, failed authorization, or unavailable route would be a setup outcome rather than a cache measurement. Keeping those categories distinct prevents a great deal of confident but uninformative benchmarking.

Authentication is not a routing guess

The ordinary API path and the optional subscription path have separate selection and funding semantics. Loading a manual does not switch the application to subscription billing. Supplying a billing mode does not authorize arbitrary account discovery.

The optional path reads an explicitly selected external credential source as data. It does not overwrite that file, renew it through an unimplemented flow, or fall back silently to an API key when it is missing or expired. Any user-supplied refresh operation remains an explicit boundary and must preserve the chosen credential-source identity. Requests carrying someone else's account identity must not survive a race between token and account selection.

These rules are not peculiar to a cache experiment. An automated assistant that quietly changes its funding route because a request failed can be operationally successful and still violate the user's decision.

Observe without taking over

The library exposes measurements through its existing Engine and Agent relationships. A minimal external consumer can select configuration, make requests, and inspect snapshots without importing internal partial implementations or creating a second accounting system.

Child and judge usage can contribute to cumulative cost while leaving the parent's latest direct-response and cache view intact. Replayed historical usage does not become a new purchase. This is the same ownership distinction that made pressure enforcement possible in Memory.

The resulting display should help an engineer ask the next useful question: did the request prefix change, did the provider report reuse, what was actually billed or estimated, and which route was selected? A single unlabeled “cache percentage” cannot answer all four.

The conversation is the principal reusable asset. Keeping its authority, ordering, and provenance coherent gives caching a chance to help without giving it permission to rewrite what the Agent knows.

Implementation chapter

Chapter 24: Gateway

In Bill's Deephold game, agents waited for their turns inside blocked tool handlers. The game needed a message router, not more elaborate waiting tools. The same design became concrete in Puffin, where a family-care application needed dependable communication across independently running programs.

Gateway moves that responsibility outside the Agent. It authenticates peers, checks routing policy, stamps provenance, and keeps messages for offline recipients. It owns no Ensemble Agent and makes no model calls. A standalone interviewer, an Ensemble adapter, and a future home-automation client can share the transport without sharing an Agent framework.

Establish identity before accepting instructions

An internet message cannot authenticate itself by saying "I am your operator." Gateway establishes identity before placing the sender's content in a model request. The operator registers peers and keys, assigns group memberships, and decides which groups a recipient accepts.

Noise-KK runs inside the WebSocket. Both endpoints already know the other's static public key: the client pins Gateway's key, and Gateway looks up the configured client key. This remains useful behind a terminating reverse proxy. TLS may protect both

network legs, but the proxy terminates those connections; the inner Noise session provides application-to-application confidentiality and authentication across it.

The client first opens `/v1/ws` with its Agent ID. That cleartext ID selects a candidate key, not a trusted identity. The two Noise handshake messages establish directional session keys while binding the application prologue and Agent ID. Encrypted JSON follows, starting with a `hello` that repeats identity and protocol version. The client is connected only after an identity-matched encrypted welcome.

The implementation preserves `Noise_KK_25519_ChaChaPoly_SHA256` through pinned Noise.NET code and libsodium. X25519 keys are agreement keys, not signing keys despite occasional historical `ed25519:` labels. The wrapper delegates the cryptography rather than inventing a compatible-looking cipher.

Key files contain private bytes; public keys are derived on load. First creation publishes a fully written private file exclusively, so two creators adopt the winner instead of each returning a different identity. A malformed existing key refuses rather than being silently replaced. Noise protects traffic, not a stolen endpoint key or plaintext queue files on disk.

A failed handshake cannot evict an existing live client. Only a fully authenticated replacement can supersede that identity's current session. Replacement revokes the old session before closing its socket, preventing buffered old requests from acting as the new connection.

Trust follows the route that admitted the message

Groups combine capability and trust. A sender needs `send-message`, and sender and recipient must share a route the recipient has explicitly accepted. Among those eligible groups, Gateway selects the strongest trust level: `owner`, `trusted`, or `external`. Membership in an unrelated owner group cannot lend owner authority to a message admitted through a weaker group.

Gateway supplies source identity, authorizing group, trust, and timestamp. A readable header naming the source, group, and trust sits above the original content. The sender cannot manufacture those authenticated fields by writing a similar header in its body. The body remains untrusted input, and the model can still mishandle it; routing admission is not a replacement for tool authority checks.

The stamp is fixed at admission and survives queueing and replay. A later membership change does not rewrite an already queued message as owner-authorized. For new calls, live policy belongs to the Router, not stale copies stored in each connection: removing a capability affects the next call from an existing session.

MCP discovery provides `initialize`, `tools/list`, and `tools/call` inside the encrypted Gateway frames. Its tools expose messaging, group listing, contactability declaration, and narrowly granted child registration. When MCP capabilities are advertised, those requests take precedence; legacy raw RPC remains a compatibility path. These are Gateway's framing choices around MCP requests, not a claim to define a new standard MCP transport.

Declaring contactability replaces the accepted groups and cannot join a new group. Registering a child requires an unused identity with a supplied public key and memberships drawn only from the parent's own. Administrator operations require the administrator capability rather than merely a familiar-looking group name. A reply still passes ordinary routing policy; having received a question does not grant a return channel by itself.

Receipt means the application accepted delivery

Every recipient has a sequenced queue. `SendAsync` succeeds when the Router authorizes and admits the message, not when the recipient has finished responding. Offline recipients accumulate queued envelopes.

On delivery, the client invokes its application's handler. Only after that handler returns successfully does it send a receipt. A handler that cannot accept work must refuse instead of acknowledging a silently dropped message. `LastAckedSeq()` advances when that receipt is successfully written, not when a notification inside the handler happens to fire.

After reconnect, the client explicitly supplies its last acknowledged sequence. Gateway replays the remaining queue. There is no hidden reconnect or retry loop. The hello receipt may recover a lost acknowledgment up to the queue's allocation frontier; later receipts can advance only through what that session actually sent. Receipts are monotonic, and draining a queue does not reset its sequence numbers.

This provides at-least-once delivery within the retained snapshot and application-acceptance boundaries. It cannot provide exactly-once application work. An application might

accept a prompt, acknowledge it, and crash before saving the resulting task. Conversely, delivery may be repeated when an acknowledgment was lost. The recipient has to define what acceptance means for its own operation.

The client's cancellation scopes stay separate. The token supplied to `ConnectAsync` owns the whole connection lifetime, not just the handshake. Canceling an individual RPC wait removes that correlation; a late response is ignored without closing the connection. `Close` interrupts transport and is safe from a callback; joining disposal waits for callbacks, reader, and cleanup. Incoming callbacks are serial, so a callback must not await an RPC response that only the same reader can dispatch, or await disposal that must join the callback itself. No background retry is implied by any of those operations.

A prompt owns its answer

The optional Ensemble bridge borrows an existing Agent. Each incoming prompt goes through `Agent.SubmitAsync`, which synchronously admits it to the actor mailbox and returns that request's `Task<string>`. A bridge-owned observer waits for that exact task before replying.

This is the important ownership boundary when the Agent is busy. Another delivery creates another logical request with its own result; it cannot steal an answer by waiting for a shared idle event or sampling a lossy transcript stream. Admission is still not synchronous History publication, and a receipt may precede model completion.

Only messages marked `async` request a reply. The response uses the authenticated sender and preserves the conversation ID without asking for another automatic response, avoiding reply loops. Turn failures are logged locally rather than shipping

exception text to the remote peer. Observers also handle the startup race where a delivery arrives before `ConnectAsync` has returned its Client; bridge readiness follows accepted contactability, not merely an open socket.

Canceling the bridge closes its connection and joins observer and client cleanup. It does not undo work already admitted to the borrowed Agent. The application separately owns Agent shutdown, saving History after the actor has joined, and Ensemble destruction. Transport lifetime and task lifetime are related, but neither silently owns the other.

Queue durability is useful and bounded

The host snapshots policy, memberships, contactability, child identities, queues, and receipt positions every thirty seconds and again on orderly shutdown. It retains the highest allocated sequence even for an empty queue, so a later message cannot reuse a receipt number.

Periodic saves leave a crash window. Individual file replacements also do not form a transaction across all queues and policy. Those limits are better understood as concrete windows than hidden behind a claim of “durable messaging.” The data directory needs protection because it contains both the private key and pending plaintext content.

Only absent state gets empty defaults. Malformed JSON, duplicate fields, or a blocked path is an error, not permission to forget queued work. Save failure leaves live state intact for a later retry and remains visible. Shutdown closes admission, joins sessions and the timer, then attempts its final save; failure must reach the application’s exit result.

The concurrency rules serve the same boundary. Socket writes never hold the policy lock. Each session serializes encryption with sending so nonce order matches frame order. A write failure after encryption terminates that session; retrying as though the nonce had not advanced would be unsafe. Pending handshakes are owned sessions too, so shutdown cannot overlook them.

The independent interviewer exercises the public client rather than reaching into Ensemble. It accepts answers only from the authenticated subject and matching conversation, and it refuses a full inbox instead of acknowledging a dropped answer. That small consumer illustrates the design's value: the router establishes identity, policy, and delivery; the application decides what a question and its answer mean.

Implementation chapter

Chapter 25 — Goals

Bill would give Ensemble a job meant to occupy most of a day, then find it idle an hour later. The model had finished a response; the job was not finished. Resuming work required a human poke. After enough context compression, that poke might also need to reconstruct why the Agent had been doing the work in the first place.

Goals separates those two needs. A file keeps the agreed objective present in each ordinary request. An optional idle reminder gives the Agent another opportunity to inspect its state. The first preserves direction; the second supplies a turn.

Keep the objective outside the conversation

A useful goal file records what remains, what counts as done, and where authority ends. It is closer to a short work agreement than a command to repeat. “Continue the migration, verify the changed paths, and stop before deployment” lets the Agent adapt to evidence without converting persistence into permission to deploy.

The `goals` skill grants `goal_control`. Loading the skill neither creates the file nor starts reminders. Ordinary file tools write and update the goal; the control selects the file, enables

reminders, and optionally changes the default eight-minute idle delay. Calling it without fields reports status without postponing a pending deadline.

Selecting a file validates it through the Agent's existing Sandbox before publishing configuration. The file must be nonblank, strictly decoded UTF-8, regular, confined, and no larger than one MiB. A bad selection leaves the previous configuration unchanged. File access uses the same authority boundary as the Agent's other file operations rather than a privileged path around it.

The path is configuration; the text stays in the file. Every ordinary provider request rereads it, including subsequent requests within the same tool loop. If the Agent updates its own goal, the next request sees the update. Skills, History, and the GUI do not maintain competing copies of the objective.

Reminders off still means instructions on

The configured goal is projected into system-side instructions even when idle reminders are disabled. On Anthropic it follows existing system blocks; chat-completions-style adapters add a system message. Responses uses the top-level `instructions` field. This is request rendering, not another History event that compression must preserve.

Auxiliary compressor, judge, and embedding requests do not receive the parent goal. They have bounded jobs of their own, not an invitation to pursue the surrounding task. Synthetic reminder text is also marked so AutoRecall does not search on it as though it were a new human question.

Disabling reminders retains the file and its instructions. Clearing the file removes configuration and injection. Unloading the skill also clears configuration, but neither operation deletes the human's file. Configuration is session-local: a fresh Agent does not restore it, and a child does not inherit the parent's goal.

If a configured file becomes unreadable, Goals disables reminders and reports a stable error. Human work may continue with an explanation; an automatic wake cannot infer without the objective that justified waking it. The read carries the configuration epoch, so a late failure for an old file cannot disable a newer goal. Off and Clear remain available even when the file is missing. Repairing the file does not secretly restart automation—the user or Agent must explicitly enable reminders again.

The timer posts mail; the actor decides

The timer callback does not call a provider, execute a tool, or append a conversation message. It posts wake mail carrying a generation token. That token becomes stale whenever the configuration or lifecycle changes, making an old scheduled callback harmless.

At the actor mailbox, the wake must still be current, enabled, granted, and idle. Active foreground work, admitted turns, controls, startup work, and direct Engine sends prevent it. A pending human turn wins over a timer that has not yet been admitted. Once the wake is admitted, it is an ordinary serialized Agent turn rather than a second loop beside the human's work.

Independent Jobs may remain alive while the actor is idle. The reminder therefore tells the model to inspect current state, continue already authorized unfinished work, leave healthy

background work alone, and disable reminders when done or blocked. An expired interval says nothing about whether a quiet child or process has failed.

The automatic turn supplies the reminder, while the goal arrives separately through the normal instructions projection. If the model does another piece of work and remains enabled, becoming idle again starts a fresh interval. This can continue a long task without requiring a scheduler service, a goal database, or a second model conversation.

It is still an opportunity, not a progress detector. A timer cannot diagnose an active stuck request, recognize task completion reliably, or expand provider, deployment, device, account, or retry authorization. Those decisions remain part of the Agent's work under the human's original boundaries.

Stopping has precise meanings

Pause suspends reminders without clearing the goal or enabled state. Resume begins a fresh interval rather than consuming the remainder of an old one. Explicit Interrupt disables reminders even if no request is active; otherwise stopping a turn could merely postpone its automatic replacement.

Shutdown and actor failure disable reminders and invalidate pending wakes. Unload and destruction invalidate queued wake mail too. A delegated Agent's successful submission also disarms reminders and invalidates pending wakes. Its retained goal may remain visible as instructions, but finishing a delegated task cannot be followed by a surprise automatic turn that reopens it.

That last boundary matters in a swarm. Submission is an observable completion transition for the parent, not merely a quiet period on the child's timer. Parent coordination would

become unreliable if a completed child could wake itself again from an earlier reminder schedule.

Show what the selected Agent accepted

The GUI's Goals controls follow the selected Agent. Apply, Reminders off, Clear, and Reload wait for actor acknowledgment; a changed checkbox is not evidence that configuration succeeded. The browser displays unloaded, unconfigured, working, paused, off, file-error, or waiting state as appropriate, including the next due time when known.

Status refreshes preserve unsaved draft edits rather than overwriting them, and Pause/Resume remains separate from disabling or clearing. A reconnect or conversation switch clears stale display state until the new snapshot arrives. The browser does not edit the goal file, maintain its own reminder timer, or persist this session-local configuration in appearance or memory preferences.

Goals closes a modest but recurring gap: the distance between finishing a response and finishing an authorized job. The objective stays current through compression, and idle time can produce a new chance to act. When the work is complete or genuinely blocked, disabling reminders is the control action; explaining that state to the human is still the Agent's responsibility.

Implementation chapter

Epilogue: What we can rebuild

Bill describes the origin of this work in a sentence: “I gave Astra a book. Astra gave me an AI coding agent.” The book carried a construction sequence, explanations, and checks. Executing it produced an application he could use. Work on a later implementation then took place inside the environment that earlier work had produced.

That is a more interesting result than a large pile of generated source. It also leaves a harder question. Which of the judgments that made the application useful can the next execution recover?

The C# work did not answer that by removing the expert. Bill corrected the architecture, challenged unnecessary mechanisms, resolved requirements, and insisted that the code remain teachable. Compilation could establish that partial declarations had matching implementations. It could not establish that the relationships were sensible, that the assistant had preserved one authoritative fact, or that the next reader could follow the cleanup.

Two late repairs show what that involvement was protecting.

The input that was already there

The memory compressor's captured input already contained the existing long-term memory. A later step appended that memory again while preparing the worker request.

Taken separately, each function looked reasonable. Capture included the information needed to understand the conversation. Request construction made sure the summarizer had the current memory. Taken together, they sent the same material twice.

A local test of either function could have reinforced the mistake. The useful test used the real capture formatter and traced the complete request. Once the duplicated input was visible, the repair was to keep one authoritative formatting path, not introduce a policy for reconciling two copies.

The same mistake can occur without an LLM anywhere nearby. One layer adds a default, another layer helpfully adds it again, and neither knows which one owns the decision. Model requests make the cost easy to observe: the duplicate consumes context and money, and may change the emphasis of the input. The structural problem is older than the API.

The chapter needs to preserve the reason for the repair. “Include memory exactly once” is a useful assertion. Knowing that capture already owns its inclusion tells the next engineer where a new feature belongs.

The task that was already done

Goals initially had local checks for file refresh, reminders, pause, interruption, and GUI controls. A delegated Agent could nevertheless publish its result and leave reminders enabled. Later, a timer could admit more work for a task whose parent had already received completion.

The missing boundary was successful submission. It was not enough to disable reminders whenever the model tried to submit: a malformed result or failed durable write leaves the task unfinished. It was not enough to wait for generic idle status: an idle Agent may still owe the parent an answer.

The correction stopped reminders only after durable publication succeeded. The actor retained responsibility for timer disposal. The goal file remained available, and the parent's configuration stayed unchanged. A follow-up check then exposed a stale status snapshot still showing the old deadline; the visible account of the task needed to agree with its new state too.

Each individual feature had a plausible implementation. Their composition lacked one important transition. The failure became understandable only by following the whole operation from the parent's request through the child's accepted result and into the later timer wake.

This is why ownership and lifecycle occupy so much of a book nominally about talking to language models. The network call is only one participant. The application also has to know what has ended, which obligations remain, and which object is allowed to make that decision.

Preserve reasons, not a legend

A passing test is evidence about the path it exercised. It is not an architectural review, a provider agreement, or a prediction that the next model will cooperate. A local fake can show that cancellation reaches the transport. It cannot establish that a commercial endpoint accepts today's credentials. A speech transcript can expose missing words without showing that a listener can comfortably understand them.

The useful response is to keep the evidence attached to the claim. Historical observations stay historical. A current experiment reports the request that was admitted, the result that was measured, and the attempts that never reached

measurement. A defect found by inspection is not promoted into a demonstrated runtime failure. An experiment that required expert correction is not presented as unattended success.

That discipline keeps the book open to improvement. It also prevents ordinary setup mistakes from acquiring the prestige of discoveries. Guessing a filename or a method signature and receiving an error consumes time; it need not reveal anything about the product. Reading the source before making the change is still cheaper.

The agentic codebook attempts to preserve enough knowledge to rebuild the system. This companion attempts to make that knowledge available to an engineer without requiring the engineer to execute every specification. Neither artifact makes judgment obsolete. They make the judgments visible enough to question and specific enough to test.

Calling generated source a secondary artifact does not lower its quality requirements. Regenerating tangled ownership or incomplete cleanup is not a useful kind of reproducibility. The result still has to be worth maintaining, and its explanations must remain accurate as it changes.

The next execution may use a better model, encounter a changed provider, or reveal an ambiguity nobody noticed. The durable contribution is a route back to the decisions: what was authoritative, what the alternative broke, what observation supported the repair, and what remained uncertain.

That is knowledge an engineer can carry into a different agent, or a different kind of program altogether.

Related account: codebook epilogue.

Source: Codebook epilogue.